CracksLatinoS! 2006

-* Concurso 111 Nivel3 *-

Programa	bLaCk-eye UnPackMe No.1
Download	FTP de Ricardo
Descripción	Ejecutable protegido
Herramientas	OllyDbg 1.10 (Parcheado 4)+ ImpREC 1.6f + RDG Packer 0.6.5 Beta + PEditor 1.7
Dificultad	Depende del nivel de cada uno..
Compresor/Compilador	Ni idea, jeje..
Protección	EXE Empakado + NagScreen!
Objetivos	Desempacar manualmente
Cracker	SyXe'05 **Fecha:** 25/11/06
Tutorial n°	7° concurso

-** Introducción **-

Muy buenas a todos los CracksLatinoS =), nos volvemos a ver las caras otra vez, jeje.. esta vez para tratar de desempakar un crackme (UnPackMe ☺)

Según los objetivos del concurso tenemos que desempakar y eliminar una NagScreen que se carga junto al crackme, lo veremos en la siguiente parte!

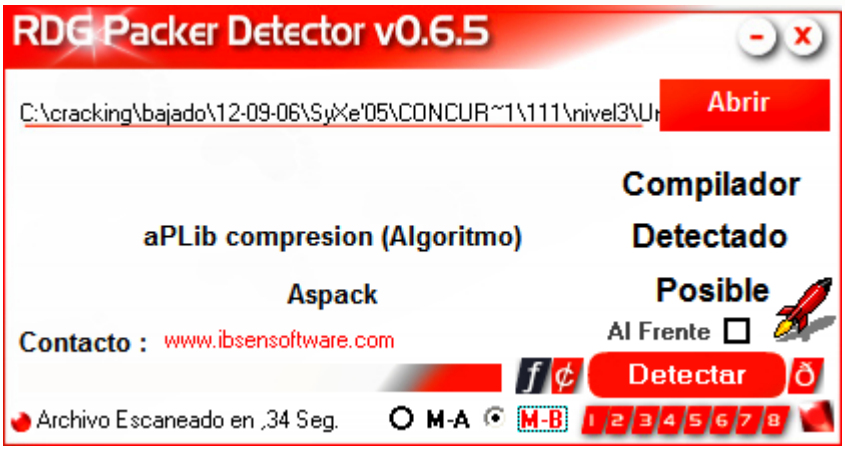
-** Explorando el objetivo **-

Desempolvemos los analizadores de archivos a ver qué ven, comenzamos con PEiD 0.94 que últimamente anda muy desacertado :(y nos dice esto:

Nothing found *

Tanto con la opción "Normal Scan" como "Deep Scan" o "Hardcore Scan" me saca esto, o sea que no ve nada raro jeje..

En vista del poco éxito utilizo el RDGPacker Detector 0.6.5 BETA, este es uno de los analizadores más potentes que conozco ya que rdgmax se toma el trabajo de actualizar sus signaturas constantemente (enhorabuena amigo=), así pues abrimos RDG Packer y lo analizamos en modo M-B a ver que encuentra:



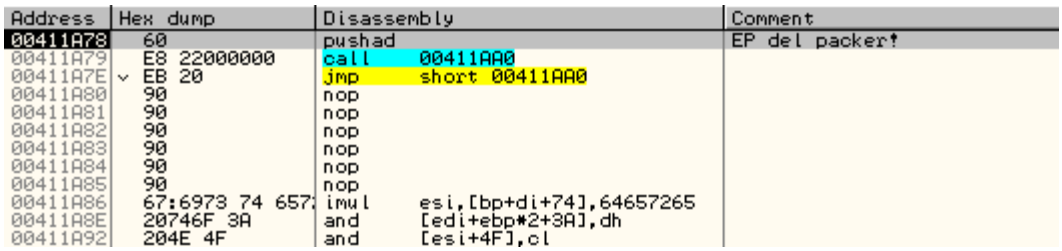
hummmmm.. aPLib compresión (Algoritmo), esto sale, bueno como tampoco sabemos de qué protección se trata es un buen comienzo, pero no nos quedemos aquí que aún hay más, pulsamos en el botoncito justo a la derecha de DETECTAR y nos muestra dos escaneos más (Externos), si elegimos el Modo M-B nos dice esto:

O sea que ExeSax 0.91 podría ser también el rival, en cualquier caso me fío muchísimo más de RDG Packer que de PeiD (incapaz de mostrar NADA), además la actualización es bien reciente (15/11/06).

Con todo esto más lo que dice el enunciado ya nos podemos meter de lleno con el packer a ver que encontramos ☺

-** Desempacando **-

Pues nada, Cargamos el EXE en Olly y para en su EntryPoint (EP):

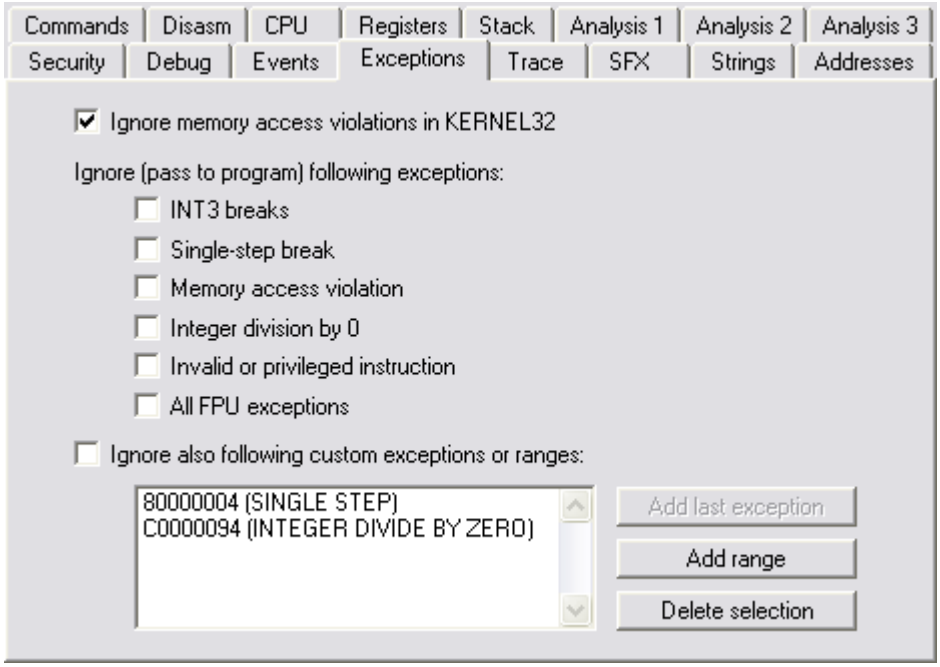


Address	Hex dump	Disassembly	Comment
00411A78	60	pushad	EP del packer!
00411A79	E8 22000000	call 00411AA0	
00411A7E	EB 20	jmp short 00411AA0	
00411A80	90	nop	
00411A81	90	nop	
00411A82	90	nop	
00411A83	90	nop	
00411A84	90	nop	
00411A85	90	nop	
00411A86	67:6973 74 657	imul esi,[bp+di+74],64657265	
00411A8E	20746F 3A	and [edi+ebp*2+3A],dh	
00411A92	204E 4F	and [esi+4F],cl	

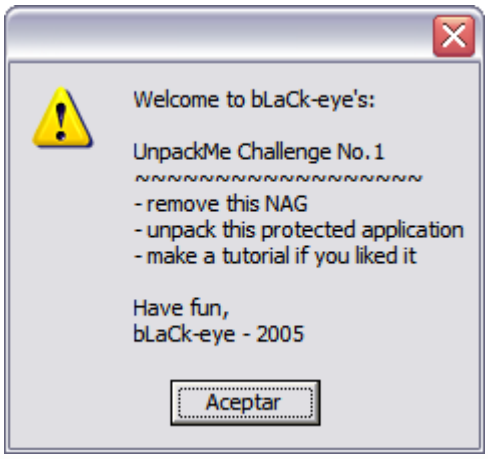
Hombre un PUSHAD jeje.. si traceamos una línea con F7/F8 vemos que ESP se vuelve de color ROJO, lo seleccionamos con el botón izquierdo del ratón y luego picamos con el derecho sobre él y elegimos FOLLOW IN DUMP, marcamos los 4 bytes que allí se ven y elegimos BREAKPOINT-HARDWARE, ON ACCESS-DWORD, pulsamos F9 y para no muy lejos:

```
Address   Hex dump        Disassembly                    Comment
00411A78  60              pushad                         EP del packer!
00411A79  E8 22000000     call    00411AA0
00411A7E  EB 20           jmp     short 00411AA0
00411A80  90              nop
00411A81  90              nop
00411A82  90              nop
00411A83  90              nop
00411A84  90              nop
00411A85  90              nop
00411A86  67:6973 74 657  imul    esi,[bp+di+74],64657265
00411A8E  20746F 3A       and     [edi+ebp*2+3A],dh
00411A92  204E 4F         and     [esi+4F],cl
00411A95  4E              dec     esi
00411A96  2D 434F4D4D     sub     eax,4D4D4F43
00411A9B  45              inc     ebp
00411A9C  52              push    edx
00411A9D  43              inc     ebx
00411A9E  49              dec     ecx
00411A9F  41              inc     ecx
00411AA0  61              popad                          El POPAD!
00411AA1  5D              pop     ebp
00411AA2  E8 00000000     call    00411AA7
00411AA7  5D              pop     ebp
00411AA8  C0E0 EC         shl     al,0EC                 Shift constant out of range
00411AAB  40              inc     eax
00411AAC  0FACC0 99       shrd    eax,eax,99             Shift constant out of range
```

Vaya por Dios! No corrimos mucho así! Bueno destildemos todas las Excepciones:



Demos RUN a ver si corre, antes de continuar os muestro mis plugins, además decir que voy con un Parcheado 4 (Evita bug de Illegal Instruction) y NO corre ;(bueno quitemos el HBP del PUSHAD anterior y..



Vale, la NagScreen a eliminar, allí dice claro además los objetivos del crackme que son:

1.- Eliminar la Nag.
2.- Desempakar la protección.
3.- Hacer un tute si puedes hacerlo ;)

Pues en ello estamos bLaCk-eye, jeje.. de momento si somos observadores veremos que el proceso principal murió:

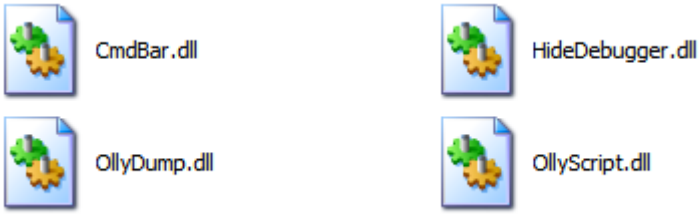
Que quiere decir esto? Lo más importante ahora es saber que el ejecutable protegido actúa como loader o cargador de la verdadera aplicación (el crackme), o sea, que hace algunas operaciones que veremos después y carga al verdadero proceso, lo que haremos es bien simple, antes de NADA es ver cómo y cuando toma la decisión de cargar al HIJO (segundo proceso), sabemos que NO lo está debuggeando porque el Proceso1 (Padre) se cerró. En caso contrario quedaría en un loop con WaitForDebugEvent, etc..

Entonces, antes de hacer cualquier otra cosa investiguemos un poco, aceptamos la Nag y nos sale el verdadero crackme:

Allí está, en inglés pone lo siguiente:

"La idea es muy simple: Eliminar la protección Anti-cracking que protege al ejecutable y obtener una versión completa desempakada"

Bien, antes de que se me olvide os muestro mis plugins de Olly ;)

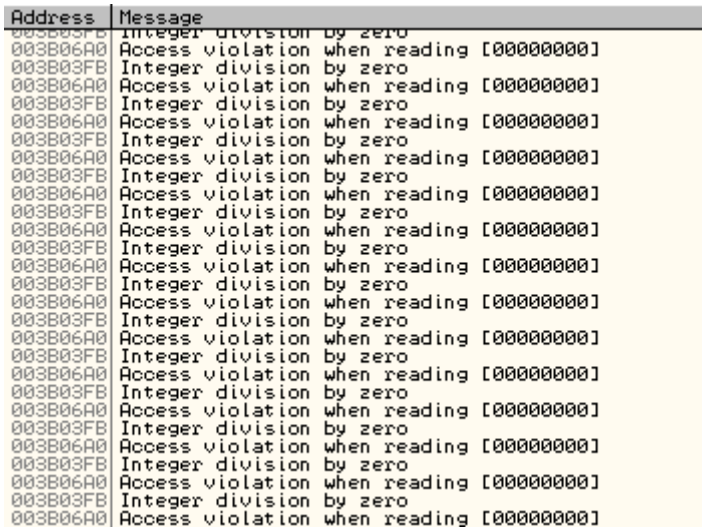


Ahí están, de momento estos cuatro y como siempre digo si no nos llega con estos ya miraremos de agregarle alguno, pero de momento sólo estos ok?

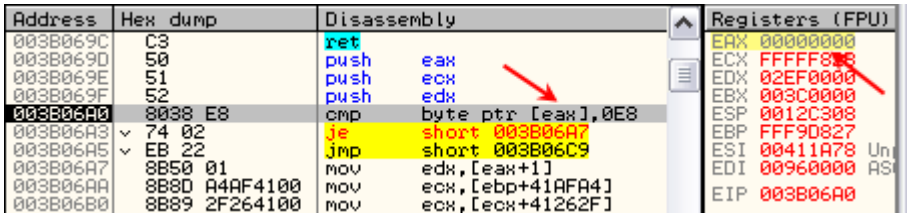
Y ahora sí a la carga, reiniciamos Olly con CTRL+F2 y ponemos un BP en la API CreateProcessA con la que se crean los procesos:

Allí está puesto, quitamos cualquier BP o HBP que tengamos puesto y demos RUN, Olly se vuelve LOCO quedando bloqueado (como suspendido) ya que el crackme no arranca ¿? Si miramos la barra de estado vemos:

Hummm.. ACCESS_VIOLATION, lo mejor de todo es que si miramos en la ventanita Log con ALT+L allí veremos lo que está sucediendo:



O sea que quedó en un loop de excepciones como se puede ver, si pausamos Olly éste se detendrá justo encima de la instrucción maligna:



Intenta acceder a [EAX] cuando EAX==0x00000000h lo que provoca una violación de acceso a memoria ya que intenta acceder a memoria protegida (sólo accesible bajo nivel Ring0 => IOCPL==0), estuve investigando y por más que hacía no salía bien parado así que cambié de táctica, jeje..

-** Cambiando de Táctica**-

Así es señores, es sabido que cuando algo NO funciona se ha de cambiar, así es la historia y puesto que el BP no funciona colocaremos un BPM ON ACCESS en dicha API a ver si así nos para (primero eliminar el BP):

Primero hacemos el GO TO (Ir a..) y ponemos el nombre de la API en cuestión (CreateProcessA en este caso) ,pulsamos OK y estamos allí. Ponemos el BPM:

```
Address   Hex dump    Disassembly                                  Comment
77E41BC0    55        push    ebp
77E41BC1    8BEC      mov     ebp,esp
77E41BC3    6A 00
77E41BC5    FF75 2C
77E41BC8    FF75 28
77E41BCB    FF75 24
77E41BCE    FF75 20
77E41BD1    FF75 1C
77E41BD4    FF75 18
77E41BD7    FF75 14
77E41BDA    FF75 10
77E41BDD    FF75 0C
77E41BE0    FF75 08
77E41BE3    6A 00
77E41BE5    E8 F60C020
77E41BEA    5D
77E41BEB    C2 2800
77E41BEE    6A 00
77E41BF0    FF7424 08
77E41BF4    E8 8E00000
77E41BF9    C2 0400
77E41BFC    8B4C24 04
77E41C00    8B5424 08
77E41C04    8B01
77E41C06    F0:0FB111
77E41C0A  ^ 75 FA
```

```
Backup                          ▸
Copy                            ▸
Binary                          ▸
Assemble                Space
Label                   :
Comment                 ;
Breakpoint                      ▸   Toggle                F2
Run trace                       ▸   Conditional           Shift+F2
                                    Conditional log       Shift+F4
New origin here         Ctrl+Gray *  Run to selection     F4
Go to                           ▸
Follow in Dump                  ▸   Memory, on access
                                    Memory, on write
Search for                      ▸
```

Una vez puesto damos RUN con F9 y Olly para aquí:

```
Address   Hex dump    Disassembly                                  Comment
003B069E    51        push    ecx
003B069F    52        push    edx
003B06A0    8038 E8   cmp     byte ptr [eax],0E8                   Es un CALL?
003B06A3  v 74 02     je      short 003B06A7
003B06A5  v EB 22     jmp     short 003B06C9
003B06A7    8B50 01   mov     edx,[eax+1]
003B06AA    8B8D A4AF4100 mov ecx,[ebp+41AFA4]
003B06B0    8B89 2F264100 mov ecx,[ecx+41262F]
003B06B6    03D1      add     edx,ecx
```

Está leyendo el primer byte de la misma y lo compara con 0x0E8h (OpCode de CALL), sigamos dando RUN y para justo debajo:

```
Address   Hex dump    Disassembly                                  Comment
003B06DD    8A10      mov     dl,[eax]                             Lee byte API
003B06DF    80FA 50   cmp     dl,50                                Es 0x50h?
003B06E2  v 72 0A     jb      short 003B06EE
003B06E4    80FA 5F   cmp     dl,5F                                Es 0x5Fh?
003B06E7  v 76 3F     jbe     short 003B0728
003B06E9    80FA CC   cmp     dl,0CC                               Hay un BP?
003B06EC  v 74 3A     je      short 003B0728
003B06EE    80FA 6A   cmp     dl,6A                                Es 0x6Ah?
003B06F1  v 74 39     je      short 003B072C
003B06F3    80FA 68   cmp     dl,68                                Es 0x68h?
003B06F6  v 74 38     je      short 003B0730
003B06F8    80FA B0   cmp     dl,0B0                               Es 0x0B0h?
003B06FB  v 72 05     jb      short 003B0702
003B06FD    80FA B7   cmp     dl,0B7                               Es 0x0B7h?
003B0700  v 76 2A     jbe     short 003B072C
003B0702    8DB5 292F4100 lea esi,[ebp+412F29]
```

Está verificando nuevamente dicho byte, incluso mira que no haya ningún BP puesto (comparación con 0x0CCh). Un poco más abajo vuelve a parar:

```
Address   Hex dump    Disassembly                                  Comment
003B0730    B2 05     mov     dl,5
003B0732    0FB6D2    movzx   edx,dl
003B0735    8A08      mov     cl,[eax]                             Lee el byte de la API y
003B0737    880B      mov     [ebx],cl                             Lo escribe a otro lugar!!
003B0739    40        inc     eax
003B073A    43        inc     ebx
003B073B    4A        dec     edx
003B073C  ^ 75 F7     jnz     short 003B0735
```

hummmmm.. esto sí parece interesante pues va a copiar el byte a otra zona¡!!! Si miramos la ventanita auxiliar nos percatamos dónde:

```
cl=55 ('U')
ds:[003C00A1]=00  <------  Copia "PUSH EBP" a 3C00A1
```

O sea que va a copiar el byte 0x55h (PUSH EBP) a la zona 0x3C00A1h, estamos en un loop que se cumple "x" veces, cuando finaliza inserta un 0xE9h (JMP) y lo reajusta dejando la zona con el siguiente aspecto:

Address	Hex dump	Disassembly	Comment
003C00A1	55	push ebp	
003C00A2	8BEC	mov ebp,esp	
003C00A4	6A 00	push 0	
003C00A6	- E9 1A1BA877	jmp kernel32.77E41BC5	
003C00AB	0000	add [eax],al	
003C00AD	0000	add [eax],al	
003C00AF	0000	add [eax],al	

Allí se ve lo que hizo, copió parte de la API y dejó un salto directo a la línea donde dejó de copiar, técnica muy común en los packers. Entonces para que nuestro BP haga efecto deberemos colocarle aquí otro, en este caso será BP común pues no creo que lo revise y si así fuera no pasaría nada pues reiniciaríamos y lo colocaríamos HBP y no BP ;)

Address	Hex dump	Disassembly	Comment
003C00A1	55	push ebp	BP puesto ;)
003C00A2	8BEC	mov ebp,esp	
003C00A4	6A 00	push 0	
003C00A6	- E9 1A1BA877	jmp kernel32.77E41BC5	
003C00AB	0000	add [eax],al	
003C00AD	0000	add [eax],al	
003C00AF	0000	add [eax],al	

Ahora sí damos RUN y para en nuestro BP, jeje.. además vemos que la zona se rellenó un poco más de lo que habíamos visto nosotros:

Address	Hex dump	Disassembly	Comment
003C00A1	55	push ebp	BP puesto ;)
003C00A2	8BEC	mov ebp,esp	
003C00A4	6A 00	push 0	
003C00A6	- E9 1A1BA877	jmp kernel32.77E41BC5	
003C00AB	55	push ebp	
003C00AC	8BEC	mov ebp,esp	
003C00AE	- E9 C09DA877	jmp kernel32.77E49E73	
003C00B3	55	push ebp	
003C00B4	8BEC	mov ebp,esp	
003C00B6	- E9 5CB7A877	jmp kernel32.77E4B817	
003C00BB	55	push ebp	
003C00BC	8BEC	mov ebp,esp	
003C00BE	56	push esi	
003C00BF	- E9 D9D2A877	jmp kernel32.77E4D39D	
003C00C4	55	push ebp	
003C00C5	8BEC	mov ebp,esp	
003C00C7	51	push ecx	
003C00C8	51	push ecx	
003C00C9	- E9 CF19A877	jmp kernel32.77E41A9D	
003C00CE	55	push ebp	
003C00CF	8BEC	mov ebp,esp	
003C00D1	- E9 7ED0A977	jmp kernel32.77E5D154	
003C00D6	0000	add [eax],al	
003C00D8	0000	add [eax],al	
003C00DA	0000	add [eax],al	
003C00DC	0000	add [eax],al	
003C00DE	0000	add [eax],al	

Zona más rellenada ;)

Hay muchos más saltos por ahí desperdigados, jeje.. y lo que nos importa a nosotros es el punto de ruptura (BP) en el que paró Olly, si nos fijamos la ejecución se detuvo en 0x3C00A1h justo donde queríamos, miremos la pila:

Address	Value	Comment
0012F1FC	003B12C2	RETURN to 003B12C2
0012F200	00960000	ASCII "C:\cracking\bajado\12-09-06\SyXe'05\concursos\1
0012F204	00141EE0	ASCII ""C:\cracking\bajado\12-09-06\SyXe'05\concursos\
0012F208	00000000	
0012F20C	00000000	
0012F210	00000000	
0012F214	00000004	
0012F218	00000000	
0012F21C	00000000	
0012F220	00411A88	UnpackMe.00411A88
0012F224	00411A78	UnpackMe.<ModuleEntryPoint>
0012F228	FFFFF82B	
0012F22C	00412003	UnpackMe.00412003
0012F230	0000004B	

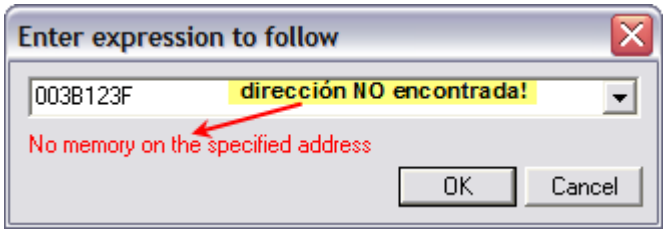
En la cima vemos el valor 0x3B12C2h, si vamos allí vemos el call:

Address	Hex dump	Disassembly	Comment
003B12AB	52	push edx	
003B12AC	6A 04	push 4	
003B12AE	6A 03	push 3	
003B12B0	59	pop ecx	
003B12B1	52	push edx	
003B12B2	^ E2 FD	loopd short 003B12B1	
003B12B4	FF95 40AF4100	call [ebp+41AF40]	
003B12BA	50	push eax	
003B12BB	57	push edi	
003B12BC	FF95 D4AE4100	call [ebp+41AED4]	
003B12C2	59	pop ecx	
003B12C3	5B	pop ebx	

En 0x3B12C2h está la llamada a CreateProcessA, jeje.. lo más interesante viene ahora pues si miramos por encima vemos esto:

Address	Hex dump	Disassembly		Comment
003B1228	E8 1EF4FFFF	call	003B064B	
003B122D	899D 04AF4100	mov	[ebp+41AF04],ebx	
003B1233	8B8D A4AF4100	mov	ecx,[ebp+41AFA4]	
003B1239	8B99 0F264100	mov	ebx,[ecx+41260F]	
003B123F	807B 03 90	cmp	byte ptr [ebx+3],90	busca un NOP!
003B1243	0F84 CC000000	je	003B1315	
003B1249	B8 00000000	mov	eax,0	
003B124E	83F8 01	cmp	eax,1	EAX==1?
003B1251	0F84 BE000000	je	003B1315	
003B1257	0399 17264100	add	ebx,[ecx+412617]	
003B125D	81C3 8B050000	add	ebx,58B	

hummmmm.. y esos saltos? vaya vaya vaya.. anotemos estas dos direcciones y reiniciemos Olly, si intentamos ir allí desde el EP veremos que no se puede:



Esto es debido a que dichas instrucciones se encuentran en memoria dinámica, lo mejor será colocar un BP en el RET de VirtualAlloc, pero ojo que el packer copia parte de la API en otra zona como ya vimos, en cualquier caso nos valdrá siempre y cuando no copie la API entera!

Address	Hex dump	Disassembly		Comment
77E48F04	55	push	ebp	
77E48F05	8BEC	mov	ebp,esp	
77E48F07	FF75 14	push	dword ptr [ebp+14]	
77E48F0A	FF75 10	push	dword ptr [ebp+10]	
77E48F0D	FF75 0C	push	dword ptr [ebp+C]	
77E48F10	FF75 08	push	dword ptr [ebp+8]	
77E48F13	6A FF	push	-1	
77E48F15	E8 92FFFFFF	call	VirtualAllocEx	
77E48F1A	5D	pop	ebp	
77E48F1B	C2 1000	ret	10	BP en el RET!
77E48F1E	8B0D 3C80EB77	mov	ecx,[77EB803C]	

Damos RUN con F9 y Olly para, miremos EAX a ver qué valor tiene:

```
Registers (FPU)
EAX 003B0000  <---- todavía nada!
ECX 77E48EF3 kernel32.77E48EF3
EDX 7FFE0304
EBX 00000000
ESP 0012F218
EBP FFFFF82B
ESI 00411D52 ASCII "^Rj@h"
EDI 77E44A4D ASCII "WaitableTimerW"
```

Esta sección 0x003B0000h que es la que buscamos, pero di vamos allí vemos que está todo a cero:

Address	Hex dump	Disassembly		Comment
003B123F	0000	add	[eax],al	No hay NADA!!
003B1241	0000	add	[eax],al	
003B1243	0000	add	[eax],al	
003B1245	0000	add	[eax],al	
003B1247	0000	add	[eax],al	
003B1249	0000	add	[eax],al	
003B124B	0000	add	[eax],al	

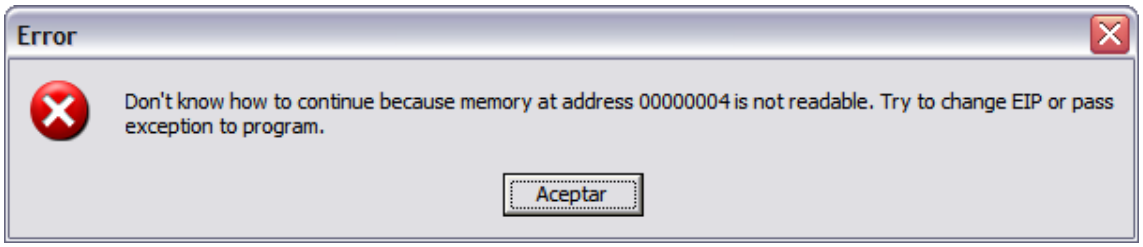
RUN nuevamente y para.. a ver ahora miremos los registros (sobre todo EAX):

```
Registers (FPU)
EAX 003C0000  <---- valor correcto!
ECX 77E48EF3 kernel32.77E48EF3
EDX 7FFE0304
EBX 003B3583
ESP 0012F218
EBP FFF9D827
ESI 003B0754
EDI 00000004
```

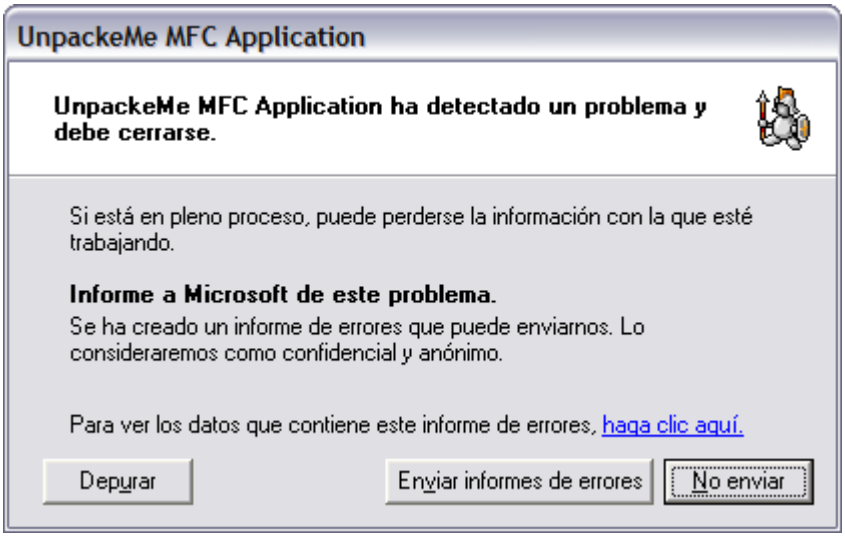
Ahí está!! En este momento tenemos los datos a la vista, hagamos el GoTo:

Address	Hex dump	Disassembly		Comment
003B1233	8B8D A4AF4100	mov	ecx,[ebp+41AFA4]	
003B1239	8B99 0F264100	mov	ebx,[ecx+41260F]	
003B123F	807B 03 90	cmp	byte ptr [ebx+3],90	BP aqui!!
003B1243	0F84 CC000000	je	003B1315	
003B1249	B8 00000000	mov	eax,0	
003B124E	83F8 01	cmp	eax,1	
003B1251	0F84 BE000000	je	003B1315	
003B1257	0399 17264100	add	ebx,[ecx+412617]	

Como se ve en la imagen colocaremos un BP con F2 en esa línea, esperemos que funcione pues como ya antes vimos los HBP los detecta y es muy probable que debamos poner un BPM ON ACCESS en lugar de BP, pero de momento lo comprobamos, jeje.. apretamos F9 para que corra y.. cataclac¡!!!



Petardazo al canto, grrrrrr.. si aceptamos..



Problemón x(por lo menos salimos de dudas, con este packer ya vemos que tanto BPs como HBPs están muy controlados así que de momento nos moveremos únicamente con BPMs.

Reiniciamos Olly con CTRL+F2 y llegamos nuevamente al RET de VirtualAlloc (VAlloc de aquí en adelante), la segunda vez que para tenemos en EAX la dirección de inicio de nuestra sección, ahí colocamos el BPM ON ACCESS:

```
Address   Hex dump     Disassembly                    Comment
003B1233    8B8D A4AF4100 mov     ecx,[ebp+41AFA4]
003B1239    8B99 0F264100 mov     ebx,[ecx+41260F]
003B123F    807B 03 90    cmp     byte ptr [ebx+3],90
003B1243  v 0F84 CC000000 je      003B1315
003B1249    B8 0
003B124E    83F8
003B1251  v 0F84
003B1257    0399
003B125D    81C3
003B1263    8B8D
003B1269    8DB1
003B126F    53
003B1270    51
003B1271    B9 5
003B1276    E8 4
003B127B    B8 0
003B1280    E8 6
003B1285    50
003B1286    68 0
003B128B    50
003B128C    6A 0
003B128E    FF95
003B1294    5F
003B1295    59
003B1296    5B
003B1297    53
003B1298    51
003B1299    8D81
003B129F    50
003B12A0    8D81
```

Backup
Copy
Binary
Assemble Space
Label :
Comment ;
Breakpoint
New origin here Ctrl+Gray *
Go to
Follow in Dump
Search for
Find references to
Analysis

Toggle F2
Conditional Shif
Conditional log Shif
Run to selection F4
Memory, on access
Memory, on write
Remove memory breakpoint

Una vez puesto damos RUN y Olly para aquí:

```
Address   Hex dump     Disassembly                    Comment
003B0442    AC          lodsb                          Para aqui!!
003B0443    32C1        xor     al,cl
003B0445    8ACD        mov     cl,ch
003B0447    8AEA        mov     ch,dl
003B0449    8AD6        mov     dl,dh
003B044B    B6 08       mov     dh,8
003B044D    66:D1EB     shr     bx,1
003B0450    66:D1D8     rcr     ax,1
003B0453  v 73 09       jnb     short 003B045E
003B0455    66:35 2083  xor     ax,8320
003B0459    66:81F3 B8ED xor    bx,0EDB8
003B045E    FECE        dec     dh
003B0460  ^ 75 EB       jnz     short 003B044D
003B0462    33C8        xor     ecx,eax
003B0464    33D3        xor     edx,ebx
003B0466    66:4F       dec     di
003B0468  ^ 75 D4       jnz     short 003B043E
003B046A    F7D2        not     edx
003B046C    F7D1        not     ecx
003B046E    5B          pop     ebx
003B046F    8BC2        mov     eax,edx
003B0471    C1C0 10     rol     eax,10
003B0474    66:8BC1     mov     ax,cx
003B0477    C3          ret
```

No es lo que buscamos ya que paró por lectura:

```
Memory breakpoint when reading [003B123F]                    Paused
```

Si nos fijamos parece como si hiciera una especie de CRC del archivo en memoria, al retornar del CALL tenemos EAX==0x7059941Ah, este valor lo compara justo después de dicha llamada, lo hace aquí:

```
Address   Hex dump     Disassembly                    Comment
003B01C5    8B8D 7F294100 mov     ecx,[ebp+41297F]    Lee CRC!
003B01CB    F9          stc
003B01CC    2BC8        sub     ecx,eax
003B01CE  v EB 03       jmp     short 003B01D3
003B01D0    CD 20       int     20
003B01D2    C7          ???                            Unknown command
003B01D3  v E3 02       jecxz   short 003B01D7         salto decisivo!!
003B01D5    4D          dec     ebp                    Problemas ;(
003B01D6    C3          ret
003B01D7    61          popad                          Todo bien ;)
003B01D8    C3          ret
```

Si miramos la ventanita auxiliar ahora veremos de dónde lee:

```
ss:[003B01A6]=7059941A   <---  CRC correcto!
ecx=0000941A             <---  dirección de memoria
```

Lo hace de [3B01A6] y allí tiene el valor 0x7059941Ah que es el mismo que hay en EAX por lo que parece ir bien ;) y llegamos al POPAD que es el sitio BUENO,

en caso de estar el archivo modificado nos llevaría al DEC EBP que es el MALO, grrrrrr.. si tiene chequeos esto¡!!!!

Si seguimos dando a F9 veremos que en lugar de parar en 0x3B123Fh lo que hace es llegar continuamente a la rutina del CRC, una posibilidad que tenemos antes de estar aquí una hora es parchear dicha rutina para que retorne el valor que sabemos debe retornar, bastaría con meterle un "MOV EAX, ValorBueno" ;)

```
Address   Hex dump    Disassembly                    Comment
003B0464    33D3      xor     edx,ebx
003B0466    66:4F     dec     di
003B0468  ^ 75 D4     jnz     short 003B043E
003B046A    F7D2      not     edx
003B046C    F7D1      not     ecx
003B046E    5B        pop     ebx
003B046F    8BC2      mov     eax,edx
003B0471    B8 1A945970  mov  eax,7059941A           valor correcto CRC!
003B0476    90        nop
003B0477    C3        ret
```

Y ahora quitamos el BPM para meterle un BP con F2 a la línea donde queremos ir, apretamos F9 y estamos allí en un santiamén, jeje..

```
Address   Hex dump    Disassembly                    Comment
003B1212    899D D0AE4100  mov     [ebp+41AED0],ebx
003B1218    E8 CEF1FFFF    call    003B03EB
003B121D    FFB5 04AF4100  push    dword ptr [ebp+41AF04]
003B1223    E8 2CF5FFFF    call    003B0754
003B1228    E8 1EF4FFFF    call    003B064B
003B122D    899D 04AF4100  mov     [ebp+41AF04],ebx
003B1233    8B8D A4AF4100  mov     ecx,[ebp+41AFA4]
003B1239    8B99 0F264100  mov     ebx,[ecx+41260F]
003B123F    807B 03 90     cmp     byte ptr [ebx+3],90     Ya estamos aqui ;)
003B1243  v 0F84 CC000000  je      003B1315
003B1249    B8 00000000    mov     eax,0
003B124E    83F8 01        cmp     eax,1
003B1251  v 0F84 BE000000  je      003B1315
003B1257    0399 17264100  add     ebx,[ecx+412617]
```

Superado el CRC seguimos, miremos la ventanita auxiliar a ver qué nos dice:

```
ds:[00400003]=00
```

Lee del PEheader!!

Así que lee de 0x400003h que forma parte del PE header como indico en la imagen, hummmmm.. que pasaría si aquí hubiera un 0x90h? probemos..

```
Address   Hex dump                                        ASCII
00400003  90 03 00 00 00 04 00 00 00 FF FF 00 00 B8 00 00  É♥...♦...  ..@..
00400013  00 00 00 00 00 40 00 00 00 00 00 00 00 00 00 00  .....@..........
00400023  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00400033  00 00 00 00 00 00 00 00 00 E0 00 00 00 0E 1F BA  .........α...ª▼│
00400043  0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68 69 73 20  ♫.┤.=↑☺L=↑This
00400053  70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F 74 20 62  program cannot b
00400063  65 20 72 75 6E 20 69 6E 20 44 4F 53 20 6D 6F 64  e run in DOS mod
```

0x90h colocado!!

Si doy RUN..

```
Command  bp CreateProcessA
Integer division by zero - use Shift+F7/F8/F9 to pass exception to program   Running
```

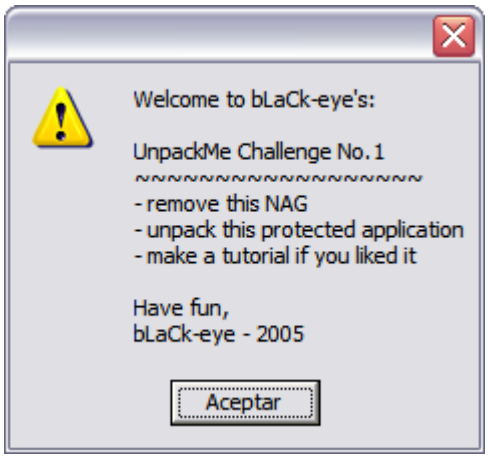
Vemos que tampoco corre, grrrrr.. Pauso Olly con F12 y para aquí:

```
Address   Hex dump    Disassembly                    Registers (FPU)
003B03F5    64:FF30   push    dword ptr fs:[eax]     EAX 00000000
003B03F8    64:8920   mov     fs:[eax],esp           ECX 0012F208
003B03FB    F7F0      div     eax                    EDX 7FFE0304
003B03FD    90        nop                            EBX 00000004
003B03FE    2BC0      sub     eax,eax                ESP 0012F200
003B0400    64:8F00   pop     dword ptr fs:[eax]     EBP FFF9D827
003B0403    83C4 04   add     esp,4                  ESI 003B35C0
003B0406    58        pop     eax                    EDI 00400000 Unp
```

Otra vez? Si esto ya lo vimos antes, de nuevo con el truquito del DIV EAX cuando EAX==0x00000000h y claro FALLA!! Salta una excepción de entero dividido entre cero y además no sigue pues parece estar en un loop. Esto lo vemos desde la ventana Log con ALT+L. Entonces lo que hago es directamente NOPEar esa línea para que Olly siga:

Address	Hex dump	Disassembly		Comment
003B03F5	64:FF30	push	dword ptr fs:[eax]	
003B03F8	64:8920	mov	fs:[eax],esp	
003B03FB	90	nop		NOPeo el DIV EAX!
003B03FC	90	nop		"
003B03FD	90	nop		
003B03FE	2BC0	sub	eax,eax	
003B0400	64:8F00	pop	dword ptr fs:[eax]	
003B0403	83C4 04	add	esp,4	
003B0406	58	pop	eax	
003B0407	C3	ret		

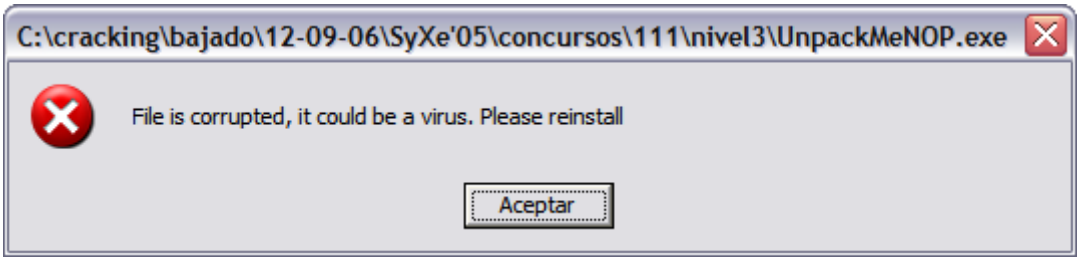
Si ahora pulso F9 veo que por fin corre.. bouuuufff!!



Y después el crackme:

Bueno algo hemos avanzado, jeje.. ya sabemos que si en la posición 0x4000003h tenemos un 0x90h (NOP) el crackme no crea al hijo, o sea que podemos debuggearla directamente, esto es bueno pero no imprescindible pues también podemos attachearnos al proceso hijo una vez creado y con la nag en pantalla, pero esta opción es igualmente correcta pues TODAS valen ;)

Pero si cambiamos el byte de [400003] a 0x90h y salvamos los cambios al disco cuando lo volvemos a ejecutar nos dice esto:



Queeeeeeeeeeee? parece que nos pillo, grrrrrr..

Si traceamos el packer llegamos a esta zona:

```
Address   Hex dump     Disassembly                          Comment
003B313E  8B8D 71594100  mov     ecx,[ebp+415971]
003B3144  56             push    esi
003B3145  51             push    ecx
003B3146  E8 2F000000    call    003B317A                     hummmmmmmmm..
003B314B  3BC7           cmp     eax,edi
003B314D  74 48          je      short 003B3197               Bonito salto ;)
003B314F  8BBD A0AF4100  mov     edi,[ebp+41AFA0]
003B3155  68 04010000    push    104
003B315A  57             push    edi
003B315B  6A 00          push    0
003B315D  FF95 18AF4100  call    [ebp+41AF18]
003B3163  8D8D 70B14100  lea     ecx,[ebp+41B170]
003B3169  6A 10          push    10
003B316B  57             push    edi
003B316C  51             push    ecx
003B316D  6A 00          push    0
003B316F  FF95 BCAE4100  call    [ebp+41AEBC]                 MsgBoxA ERROR!!
003B3175  E8 93D4FFFF    call    003B060D
003B317A  55             push    ebp
003B317B  8BEC           mov     ebp,esp
003B317D  56             push    esi
003B317E  8B4D 0C        mov     ecx,[ebp+C]
003B3181  8B55 08        mov     edx,[ebp+8]
003B3184  D1E9           shr     ecx,1
003B3186  33C0           xor     eax,eax
003B3188  F8             clc
003B3189  66:13444A FE   adc     ax,[edx+ecx*2-2]
003B318E  49             dec     ecx
003B318F  75 F8          jnz     short 003B3189
003B3191  1345 0C        adc     eax,[ebp+C]
003B3194  5E             pop     esi
003B3195  C9             leave
003B3196  C3             ret
003B3197  B8 0000A900    mov     eax,0A90000
003B319C  E8 5DD4FFFF    call    003B05FE
003B31A1  8B85 A0AF4100  mov     eax,[ebp+41AFA0]
003B31A7  E8 52D4FFFF    call    003B05FE
003B31AC  8D85 B4AF4100  lea     eax,[ebp+41AFB4]
```

En 0x3B316Fh tenemos el MsgBox de error, si saltamos en el JE de 0x3B314Dh el crackme arranca, jeje.. pero claro esto es memoria virtual que no existirá desde el inicio por lo que deberemos hacerlo cada vez.

Recapitulando un poco pongamos los pasos seguidos para vencer al CRC (físico y en memoria), son estos:

1.- Poner un BP en el RET de VirtualAlloc
2.- La segunda parada NOPear el DIV EAX (0x3B03FBh en mi máquina)
3.- Modificar la rutina del CRC (0x3B0471h)
4.- Evitar la detección (saltar en 0x3B314Dh)

Pero ojo que si traceamos nos topamos con más problemas:

```
Address   Hex dump     Disassembly                          Comment
003B1061  8995 92354100  mov     [ebp+413592],edx
003B1067  8D85 59384100  lea     eax,[ebp+413859]
003B106D  50             push    eax
003B106E  FF95 14AF4100  call    [ebp+41AF14]
003B1074  BB FFFFFFFF    mov     ebx,-1
003B1079  CD 68          int     68
003B107B  CC             int3                                 4 EXCEPCIONES¡!!!!
003B107C  CD 2E          int     2E
003B107E  CD 46          int     46
003B1080  8B4424 04      mov     eax,[esp+4]
```

Anda que pone solo una, grrrrrr..

La primera de ellas int68h se utilizaba como antidebug bajo SoftICE:

68h - Real Mode Debugger services:

SoftICE handles the following services (previous vector is hained):
43h: D386_Identify, SoftICE returns AX=0F386h.
44h: D386_Prepare_PMode, SoftICE returns a pointer to its protected mode initialisation callback function (PMINIT). This function implements services PMINIT_INIT_IDT, PMINIT_INIT_PAGING, PMINIT_INIT_SPARE_PTE and PMINIT_GET_SIZE_PHYS.
5080h and 5081h: D386_Load_Segment (device driver code/data segment).
9000h: SoftICE specific (SIWVID).

Y la detección se efectuaba así:

```
MOV AH, 43h
INT 68h
CMP AX,0F386h
JZ SoftIce_Detected!
```

Lo que no sé cómo afectará esto bajo Olly, de momento colocaremos un BPM en la sección donde nos encontramos 0x003B0000 de 9000h btyes (colocamos un BPM ON MEMORY ACCESS), damos RUN (F9) y para aquí:

```
Address   Hex dump     Disassembly                    Comment
003B106D  50           push   eax
003B106E  FF95 14AF4100 call  [ebp+41AF14]
003B1074  BB FFFFFFFF  mov    ebx,-1
003B1079  CD 68        int    68
003B107B  CC           int3
003B107C  CD 2E        int    2E
003B107E  CD 46        int    46
003B1080  8B4424 04    mov    eax,[esp+4]             Para aqui!!
003B1084  8B40 04      mov    eax,[eax+4]
003B1087  60           pushad
003B1088  E8 00000000  call   003B108D
003B108D  5D           pop    ebp
```

Justo debajo de las cuatro excepciones, jeje.. esto es bueno saberlo para futuros traceos, pues ya podríamos colocar directamente un BP en esa línea y dar RUN, jeje.. :D

Pasadas las excepciones ponemos un BP en el salto a invertir:

```
003B314D    /74 48          je      short 003B3197
```

Cuando pare modificamos !ZF (Flag Zero) y ponemos gran atención aquí:

```
Address   Hex dump     Disassembly                    Comment
003B323B  6A 00        push   0
003B323D  50           push   eax
003B323E  FF95 B8AE4100 call  [ebp+41AEB8]
003B3244  32C0         xor    al,al
003B3246  AE           scasb
003B3247  ^ 75 FD      jnz    short 003B3246
003B3249  803F CC      cmp    byte ptr [edi],0CC      cuidado!!
003B324C  ^ 75 DF      jnz    short 003B322D
003B324E  v EB 03      jmp    short 003B3253
```

Si miramos la ventana de registros vemos:

```
Registers (FPU)
EAX 00000000
ECX 77F4166A ntdll.77F4166A
EDX 00140608
EBX 003C01CD
ESP 0012F230
EBP FFF9D827
ESI 00019066
EDI 003B8851 ASCII "URSoft W32Dasm Ver 8.93 Program Disassembler/Debugger"
EIP 003B3249
```

hummm.. buscando el WDasm, jeje.. si hacemos un FOLLOW IN DUMP-MEMORY ADDRESS podremos ver de dónde está leyendo:

```
Address   Hex dump                                        ASCII
003B8851  55 52 53 6F 66 74 20 57 33 32 44 61 73 6D 20 56  URSoft W32Dasm V
003B8861  65 72 20 38 2E 39 33 20 50 72 6F 67 72 61 6D 20  er 8.93 Program
003B8871  44 69 73 61 73 73 65 6D 62 6C 65 72 2F 44 65 62  Disassembler/Deb
003B8881  75 67 67 65 72 00 4E 75 4D 65 67 61 20 53 6F 66  ugger.NuMega Sof
003B8891  74 49 43 45 20 4C 6F 61 64 65 72 00 52 65 67 69  tICE Loader.Regi
003B88A1  73 74 72 79 20 4D 6F 6E 69 74 6F 72 20 2D 20 53  stry Monitor - S
003B88B1  79 73 69 6E 74 65 72 6E 61 6C 73 3A 20 77 77 77  ysinternals: www
003B88C1  2E 73 79 73 69 6E 74 65 72 6E 61 6C 73 2E 63 6F  .sysinternals.co
003B88D1  6D 00 46 69 6C 65 20 4D 6F 6E 69 74 6F 72 20 2D  m.File Monitor -
003B88E1  20 53 79 73 69 6E 74 65 72 6E 61 6C 73 3A 20 77   Sysinternals: w
003B88F1  77 77 2E 73 79 73 69 6E 74 65 72 6E 61 6C 73 2E  ww.sysinternals.
003B8901  63 6F 6D 00 74 72 77 32 30 30 30 00 4F 6C 6C 79  com.trw2000.Olly
003B8911  44 62 67 00 4F 6C 6C 79 44 62 67 20 2D 20 5B 43  Dbg.OllyDbg - [C
003B8921  50 55 5D 00 CC 46 6F 72 20 73 61 66 65 74 79 20  PU].╠For safety
003B8931  72 65 61 73 6F 6E 73 2C 20 70 72 6F 67 72 61 6D  reasons, program
003B8941  20 63 61 6E 6E 6F 74 20 72 75 6E 20 69 6E 20 74   cannot run in t
003B8951  68 69 73 20 65 6E 76 69 72 6F 6E 65 6D 65 6E 74  his environement
003B8961  21 0D 0A 20 54 68 65 20 4F 53 20 69 73 20 63 6F  !.. The OS is co
```

Ahí están todas, parece que busca programas aplicados al cracking! Seguimos traceando y vemos como se va automodificando en loops como este:

```
003B3285     FF0C01          dec     dword ptr [ecx+eax]
003B3288     49              dec     ecx
003B3289   ^ 75 FA           jnz     short 003B3285
```

Pero ojo que viene algo mas!!

```
Address   Hex dump        Disassembly                    Comment
003B32B2    0F010B        sidt    [ebx]                  OJO!!
003B32B5    8B73 02       mov     esi,[ebx+2]
003B32B8    83C6 08       add     esi,8
003B32BB    8B1E          mov     ebx,[esi]
003B32BD    53            push    ebx
003B32BE    56            push    esi
003B32BF    83C6 10       add     esi,10
003B32C2    8B36          mov     esi,[esi]
003B32C4    81E6 FFFF0000 and     esi,0FFFF
003B32CA    81E3 FFFF0000 and     ebx,0FFFF
003B32D0    2BF3          sub     esi,ebx
003B32D2    83FE 1E       cmp     esi,1E
003B32D5    B0 90         mov     al,90
003B32D7  v 75 1F         jnz     short 003B32F8
003B32D9  ^ E9 2FD3FFFF   jmp     003B060D
```

Esa instrucción genera una excepción, hacemos como antes: BPM en la sección donde estamos y veremos donde para:

```
Address   Hex dump        Disassembly                    Comment
003B32DE    8B6424 08     mov     esp,[esp+8]            Retorna aqui!!
003B32E2    64:8F05 000000( pop   dword ptr fs:[0]
003B32E9    83C4 04       add     esp,4
003B32EC    E8 00000000   call    003B32F1
003B32F1    5D            pop     ebp
003B32F2    81ED CA5A4100 sub     ebp,415ACA
003B32F8    64:A1 30000000 mov    eax,fs:[30]
003B32FE  v EB 03         jmp     short 003B3303
003B3300    CD 20         int     20
003B3302    EA 50E80100 00( jmp   far 0000:0001E850      Far jump
```

Seguimos.. pero no muy rápido pues vienen más baches:

```
Address   Hex dump        Disassembly                    Comment
003C0087  - E9 FE87A877   jmp     kernel32.GetTickCount  Cuidado!
003C008C    55            push    ebp
003C008D    8BEC          mov     ebp,esp
003C008F  - E9 429CA877   jmp     kernel32.77E49CD6
003C0094  - E9 EFD2A877   jmp     kernel32.GetCommandLineA
003C0099    55            push    ebp
```

Ahí están, las llamadas a GetTickCount que actúan como contador de tiempo, es posible colocar un par de ellos estratégicamente como por ejemplo al inicio y final de una rutina, tomar los tiempos y calcular, si son muy mayores.. jeje.. script a la vista o me están traceando :D adiós, grrrrrr..

Viene otro loop desencriptador:

```
003B3475     FF0C01          dec     dword ptr [ecx+eax]
003B3478     49              dec     ecx
003B3479   ^ 75 FA           jnz     short 003B3475
```

Sigo y para dos veces en CreateThread que amula a partir de 0x3C009Ch en mi máquina, después viene este bucle:

```
Address   Hex dump        Disassembly                    Comment
003B3888    FFB5 00AF4100 push    dword ptr [ebp+41AF00]
003B388E    E8 3BCCFFFF   call    003B04CE
003B3893    61            popad
003B3894    E8 52CBFFFF   call    003B03EB               Call Excepcion!!
003B3899    80BD 4EAC4100 cmp     byte ptr [ebp+41AC4E],0
003B38A0  ^ 74 F2         je      short 003B3894
003B38A2    037F 3C       add     edi,[edi+3C]
```

Dentro del call está el DIV EAX NOPpeado, jeje.. entretanto crea otro hilo más, luego llama a VirtualProtect con estos parámetros:

```
0012F1E4   FFFFFFFF  |hProcess = FFFFFFFF
0012F1E8   004000E6  |Address = UnpackMe.004000E6
0012F1EC   00000002  |Size = 2
0012F1F0   00000004  |NewProtect = PAGE_READWRITE
0012F1F4   0012F206  \pOldProtect = 0012F206
```

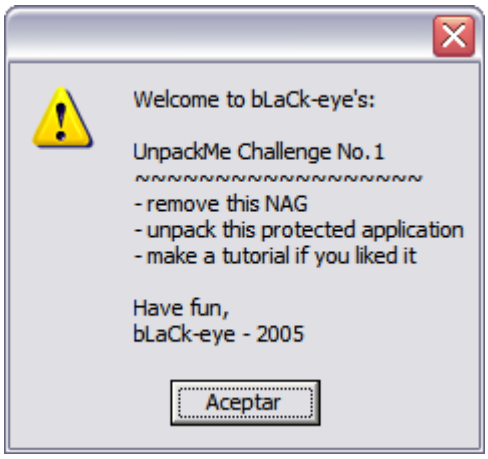
Genera otra excepción más, coloco el BPM y para aquí:

Address	Hex dump	Disassembly	Comment
003B391E	8B6424 08	mov esp,[esp+8]	Sigue aqui!
003B3922	64:8F05 000000(	pop dword ptr fs:[0]	
003B3929	83C4 04	add esp,4	
003B392C	61	popad	
003B392D	5D	pop ebp	
003B392E	60	pushad	
003B392F	8D85 55614100	lea eax,[ebp+416155]	

Luego genera otra más y salta a GetTickCount de nuevo (dos veces) y luego..

Address	Hex dump	Disassembly	Comment
003B3F9D	5D	pop ebp	
003B3F9E	85C0	test eax,eax	
003B3FA0	74 0C	je short 003B3FAE	
003B3FA2	FFD0	call eax	kernel32.IsDebuggerPresent
003B3FA4	85C0	test eax,eax	
003B3FA6	0F85 61C6FFFF	jnz 003B060D	
003B3FAC	EB 00	jmp short 003B3FAE	

Desde aquí ponemos el BPM en la sección CODE, damos RUN y sale la nag:



La acepto y para aquí:

Address	Hex dump	Disassembly	Comment
003B4200	F3:A4	rep movsb	leyendo..
003B4202	80BD 7D694100	cmp byte ptr [ebp+41697	
003B4209	0F85 86000000	jnz 003B4295	
003B420F	60	pushad	
003B4210	8D8D 416A4100	lea ecx,[ebp+416A41]	
003B4216	33D2	xor edx,edx	
003B4218	52	push edx	
003B4219	6A 20	push 20	
003B421B	6A 02	push 2	
003B421D	52	push edx	

Ahí paró, vemos el código en ROJO porque se automodificó mientras ejecutaba código (en tiempo de ejecución), este packer además de comprimir encripta las secciones (incluida la del propio packer), pero cuidado otra vez!!

Address	Hex dump	Disassembly	Comment
003B42B4	52	push edx	
003B42B5	0F31	rdtsc	RDTSC!!
003B42B7	33D2	xor edx,edx	
003B42B9	6A 05	push 5	
003B42BB	F73424	div dword ptr [esp]	
003B42BE	8F0424	pop dword ptr [esp]	
003B42C1	92	xchg eax,edx	
003B42C2	5A	pop edx	

Los RDTSC que retornan el TimeDateStamp en EAX, no sé bien explicar el porqué de ella, pero si sé que se utiliza como antidebug (y antitraceo).

NOTA: Hay veces en que el programa termina ¿? hummm.. está claro que nos ha pillado mientras traceábamos o quizá algo que hicimos mal.

Lo mejor es dejar EAX==0 tras pasar por cada RDTSC y GetTickCount pues quizá fue motivado por eso. En cualquier caso hay que pasar esto para llegar adónde necesitamos, viene otro loop cuya salida está aquí:

Address	Hex dump	Disassembly	Comment
003B6D7C	^ 72 F2	jb short 003B6D70	
003B6D7E	C3	ret	
003B6D7F	2B7C24 28	sub edi,[esp+28]	
003B6D83	897C24 1C	mov [esp+1C],edi	
003B6D87	61	popad	Salimos por aqui!
003B6D88	C3	ret	
003B6D89	55	push ebp	
003B6D8A	8BEC	mov ebp,esp	

Ponemos un BP en el POPAD, damos RUN y ya podemos continuar la marcha, pero nos encontramos con un bucle MAYOR que engloba a este, salimos por aquí:

Address	Hex dump	Disassembly	Comment
003B4750	80C1 08	add cl,8	
003B4753	C3	ret	
003B4754	90	nop	
003B4755	90	nop	
003B4756	90	nop	
003B4757	90	nop	
003B4758	58	pop eax	Salida loop mayor..
003B4759	8DB5 DF6D4100	lea esi,[ebp+416DDF]	
003B475F	833E 00	cmp dword ptr [esi],0	
003B4762	v 74 24	je short 003B4788	
003B4764	8B06	mov eax,[esi]	
003B4766	8BD8	mov ebx,eax	
003B4768	8B4E 04	mov ecx,[esi+4]	

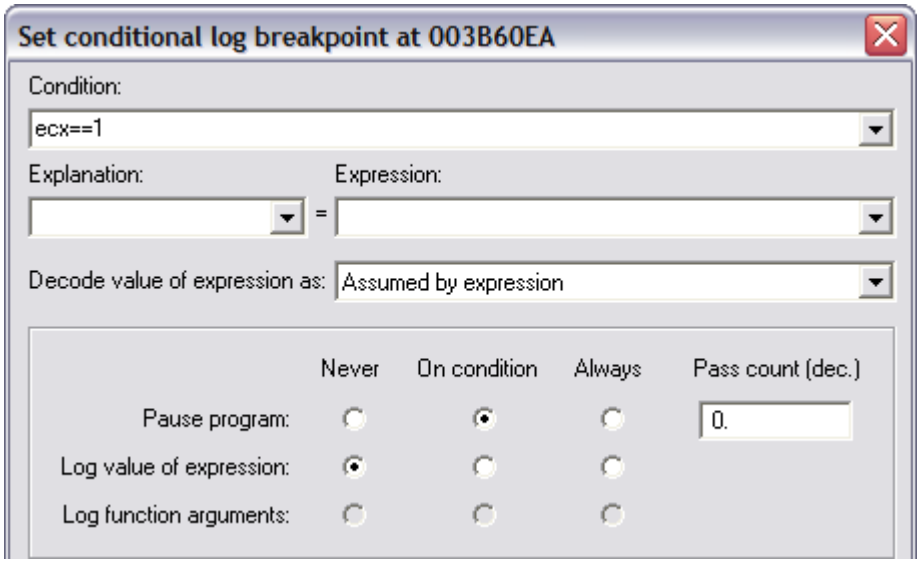
Ponemos el BP, damos RUN y al parar allí lo quitamos (F2), llegamos a otro pequeño loop desencriptador:

```
003B6037     FF0C01          dec     dword ptr [ecx+eax]
003B603A     49              dec     ecx
003B603B   ^ 75 FA           jnz     short 003B6037
```

Pero no nos vayamos muy lejos pues hay otro:

```
003B60E6     FF0C01          dec     dword ptr [ecx+eax]
003B60E9     49              dec     ecx
003B60EA   ^ 75 FA           jnz     short 003B60E6
```

Es muy pequeño el tramo a desencriptar y lo podemos hacer a mano, en caso que fuese mayor ponemos un BPCL en el JNZ de esta manera:



Y daríamos RUN, de esta manera pararía al quedar sólo 1 byte por desencriptar :D pero justo viene otro, bouuuuuuuuff..

```
003B615F     FF0C01          dec     dword ptr [ecx+eax]
003B6162     49              dec     ecx
003B6163   ^ 75 FA           jnz     short 003B615F
```

Hacemos lo mismo y seguimos.. pero cuidado aquí:

```
Address   Hex dump     Disassembly                          Comment
003B61F2    32C0       xor     al,al
003B61F4    AE         scasb
003B61F5  ^ 75 FD      jnz     short 003B61F4
003B61F7    58         pop     eax
003B61F8    803F 25    cmp     byte ptr [edi],25            cuidado!!
003B61FB  v 74 15      je      short 003B6212
003B61FD  ^ E2 DB      loopd   short 003B61DA
```

Busca el valor 0x25h en ASCII "%" dentro de estas cadenas ;)

```
Address   Hex dump                                         ASCII
003B6BC1  75 F7 C3 55 53 45 52 33 32 00 47 44 49 33 32 00  u÷├USER32.GDI32.
003B6BD1  4B 45 52 4E 45 4C 33 32 00 53 48 45 4C 4C 33 32  KERNEL32.SHELL32
003B6BE1  00 43 4F 4D 43 54 4C 33 32 00 41 44 56 41 50 49  .COMCTL32.ADVAPI
003B6BF1  33 32 00 43 4F 4D 44 4C 47 33 32 00 57 53 4F 43  32.COMDLG32.WSOC
003B6C01  4B 33 32 00 25 55 50 6A 00 B8 78 56 34 12 FF D0  K32.%UPj.╕xV4↑ ╨
003B6C11  B8 78 56 34 12 E8 00 00 00 00 5D 81 ED F4 93 41  ╕xV4↑Φ....]üφ¶ôA
003B6C21  00 35 45 23 89 06 89 85 09 94 41 00 58 5D 68 00  .5E#ë♠ëà.öA.X]h.
003B6C31  00 00 00 E8 00 00 00 00 58 89 48 F7 C3 55 8B EC  ...Φ....XëH÷├Uïÿ
003B6C41  51 B9 00 01 00 00 8B 75 0C 8B 55 08 8B 12 AD 33  Q╣.0..ïu.ïU◘ï↑↓3
003B6C51  C1 03 D0 F7 D2 D3 C2 49 75 F4 8B C2 8B 55 08 89  ┴♥╨÷╥╙├Iu¶ï┬ïU◘ë
003B6C61  02 59 C9 C2 08 00 00 00 00 00 00 00 00 A9 F3 78  ☻Y╔┬◘.......@⁴x
003B6C71  00 00 00 00 00 C9 7F E6 2A 90 90 90 8D 05 C2 94  .....╔⌂µ*ÉÉÉì♣┬ö
```

Que son los nombres de librerías de sistema donde se encuentran muchas APIs, cuando la encuentra la carga con LoadLibraryA aquí:

```
Address   Hex dump     Disassembly                          Registers (MMX)
003C0000  - E9 B0CFA877  jmp     kernel32.LoadLibraryA     EAX 004162C0 ASCII "WINSPOOL.DRV"
003C0005    55           push    ebp                       ECX 00000000
003C0006    8BEC         mov     ebp,esp                   EDX 00400000 UnpackMe.00400000
003C0008    6A FF        push    -1                        EBX 004162C0 ASCII "WINSPOOL.DRV"
003C000A    68 B0F3E877  push    77E8F3B0                  ESP 0012F210
003C000F  - E9 5329A977  jmp     kernel32.77E52967         EBP FFF9D827
003C0014  - E9 0BE89777  jmp     User32.MessageBoxA        ESI 0041505C UnpackMe.0041505C
003C0019    33C0         xor     eax,eax                   EDI 0041209C UnpackMe.0041209C
```

En este caso le tocó el turno a WINSPOOL.DRV, si seguimos un poco llegamos al call que muestra la nag inicial:

```
Address   Hex dump     Disassembly                          Comment
003B4119    68 30200000  push    2030
003B411E    8D85 E5674100  lea     eax,[ebp+4167E5]
003B4124    50           push    eax
003B4125    8D85 E7674100  lea     eax,[ebp+4167E7]
003B412B    50           push    eax
003B412C    6A 00        push    0
003B412E    FF95 BCAE4100  call    [ebp+41AEBC]             NagScreen!!
003B4134    8D85 64694100  lea     eax,[ebp+416964]
```

Pero si seguimos traceando genera una excepcion que nos lleva a:

```
Address   Hex dump     Disassembly                          Comment
003B418B    8B6424 08    mov     esp,[esp+8]                Desvio!!
003B418F    83C4 04      add     esp,4
003B4192    33C0         xor     eax,eax
003B4194    64:8F00      pop     dword ptr fs:[eax]
003B4197    E8 00000000  call    003B419C
003B419C    5D           pop     ebp
003B419D    81ED 75694100  sub     ebp,416975
003B41A3    B8 01000000  mov     eax,1
```

Es una especie de "laberinto" donde trata de perdernos, jeje.. por su puesto no le dejamos claro, pero a partir de aquí seguir hasta el OEP me fue casi imposible, grrrrrr.. intenté casi de todo y no había manera, lo que pasa es que el packer quedaba rulando en este bucle:

```
Address   Hex dump     Disassembly                          Comment
003B41CE    E8 FBC2FFFF  call    003B04CE
003B41D3    50           push    eax
003B41D4    FF95 F8AE4100  call    [ebp+41AEF8]
003B41DA    E8 0CC2FFFF  call    003B03EB                   Queda aqui ??
003B41DF    80BD F0AC4100  cmp     byte ptr [ebp+41ACF0],0
003B41E6  ^ 74 F2        je      short 003B41DA
003B41E8    A0 659FFF0E  mov     al,[EFF9F65]
003B41ED    48           dec     eax
003B41EE    BC 66AE4C47  mov     esp,474CAE66
003B41F3    D5 E5        aad     0E5
003B41F5    36:6B9F 292AF1  imul    ebx,ss:[edi+82F12A29],-2
003B41FD    F3:          prefix rep:                        Superfluous prefix
003B41FE    B3 49        mov     bl,49
003B4200    7B 5B        jpo     short 003B425D
003B4202    0BFF         or      edi,edi
003B4204    9A 4C5E749D 9A  call    far 059A:9D745E4C       Far call
003B420B    AC           lodsb
```

Esa llamada va allí donde NOPeamos el DIV EAX, quizá esa excepción haga saltar al SEH y desde allí nos desvíe, no lo sé, pues quité los NOP pero no hacía nada, un callejón sin salida grrrrr..

Entonces aquí me acordé que los crackme (como todo programa) suelen hacer uso de ciertas funciones justo después del OEP, de las más comunes que yo conozco son GetModuleHanldeA y GetVersion, con la primera probé y paraba muchas veces, no así cuando probé con la segunda, os pongo en situación:

Estamos en el EP y hacemos:

1.- GoTo => VirtualAlloc y ponemos un BP en el RET:

Address	Hex dump	Disassembly		Comment
77E48F04	55	push	ebp	
77E48F05	8BEC	mov	ebp,esp	
77E48F07	FF75 14	push	dword ptr [ebp+14]	
77E48F0A	FF75 10	push	dword ptr [ebp+10]	
77E48F0D	FF75 0C	push	dword ptr [ebp+C]	
77E48F10	FF75 08	push	dword ptr [ebp+8]	
77E48F13	6A FF	push	-1	
77E48F15	E8 92FFFFFF	call	VirtualAllocEx	
77E48F1A	5D	pop	ebp	
77E48F1B	C2 1000	ret	10	BP en el RET!
77E48F1E	8B0D 3C80EB77	mov	ecx,[77EB803C]	

2.- Al llegar allí la segunda vez pongo un BP en el salto que decide cerrar el programa y que antes descubrimos:

Address	Hex dump	Disassembly		Comment
003B3144	56	push	esi	
003B3145	51	push	ecx	
003B3146	E8 2F000000	call	003B317A	
003B314B	3BC7	cmp	eax,edi	
003B314D	74 48	je	short 003B3197	BP aqui!!
003B314F	8BBD A0AF4100	mov	edi,[ebp+41AFA0]	
003B3155	68 04010000	push	104	

NOTA:Este salto debe saltar.

3.- NOPear el DIV EAX:

Address	Hex dump	Disassembly		Comment
003B03F5	64:FF30	push	dword ptr fs:[eax]	
003B03F8	64:8920	mov	fs:[eax],esp	
003B03FB	90	nop		NOPeo el DIV EAX!
003B03FC	90	nop		"
003B03FD	90	nop		
003B03FE	2BC0	sub	eax,eax	
003B0400	64:8F00	pop	dword ptr fs:[eax]	
003B0403	83C4 04	add	esp,4	
003B0406	58	pop	eax	
003B0407	C3	ret		

4.- Evitar la detección de CRC:

Address	Hex dump	Disassembly		Comment
003B0464	33D3	xor	edx,ebx	
003B0466	66:4F	dec	di	
003B0468	75 D4	jnz	short 003B043E	
003B046A	F7D2	not	edx	
003B046C	F7D1	not	ecx	
003B046E	5B	pop	ebx	
003B046F	8BC2	mov	eax,edx	
003B0471	B8 1A945970	mov	eax,7059941A	valor correcto CRC!
003B0476	90	nop		
003B0477	C3	ret		

Y ahora sí damos RUN, cuando pare en 0x3B314Dh cambiamos el !ZF para que SALTE de esta manera evitamos que se cierre el crackme con el MsgBox ese de corrupt.. blablabla, ok?

Llegados aquí ponemos un BP en el call de la nag:

Address	Hex dump	Disassembly		Comment
003B411E	8D85 E5674100	lea	eax,[ebp+4167E5]	
003B4124	50	push	eax	
003B4125	8D85 E7674100	lea	eax,[ebp+4167E7]	
003B412B	50	push	eax	
003B412C	6A 00	push	0	
003B412E	FF95 BCAE4100	call	[ebp+41AEBC]	Llegar aqui!!
003B4134	8D85 64694100	lea	eax,[ebp+416964]	
003B413A	50	push	eax	

Quitamos TODOS los BPs que hayamos puesto y ponemos uno en GetVersion:

```
Command  bp GetVersion
```

Ahora sí damos RUN con F9, cuando pare miramos la pila:

```
Address   Value     Comment
0012E7E8  76F212D5  RETURN to WLDAP32.76F212D5 from kernel32.GetVersion
0012E7EC  77F69005  ntdll.77F69005
0012E7F0  77F5D5E0  ntdll.77F5D5E0
0012E7F4  FFFFFFFF
0012E7F8  77F417B2  RETURN to ntdll.77F417B2 from ntdll.77F68C4E
0012E7FC  77F42877  RETURN to ntdll.77F42877 from ntdll.RtlAllocateHeap
```

NADA no retorna al CrackMe!!!

Allí vemos que retornará a WLDAP32.DLL así que no nos vale, necesitamos que vuelve al CrackMe o a alguna de sus secciones, RUN nuevamente y..

```
Address   Value     Comment
0012F198  003F0221  RETURN to 003F0221 from kernel32.GetVersion
0012F19C  F352842D
0012F1A0  8320F372
0012F1A4  62362DD6
0012F1A8  0012F1F0
0012F1AC  00101000
0012F1B0  77E48EF3  RETURN to kernel32.77E48EF3 from kernel32.77E41C76
0012F1B4  00412018  UnpackMe.00412018
```

Esta SI nos vale ;)

Esta SI que nos vale, jeje.. pasamos la API con CTRL+F9 y caemos aquí:

```
Address   Hex dump     Disassembly                  Comment
003F0221    33D2       xor     edx,edx              hummm...
003F0223  ^ E9 9AFEFFFF jmp     003F00C2
003F0228  v E9 35000000 jmp     003F0262
003F022D  ^ E9 8BFEFFFF jmp     003F00BD
003F0232  ^ E9 3EFFFFFF jmp     003F0175
003F0237  ^ E9 65FFFFFF jmp     003F01A1
003F023C  v E9 48000000 jmp     003F0289
003F0241    50         push    eax
003F0242  ^ E9 56FEFFFF jmp     003F009D
003F0247  ^ E9 19FFFFFF jmp     003F0165
003F024C  ^ E9 9EFFFFFF jmp     003F01EF
003F0251    6A FF      push    -1
003F0253    68 50354100 push    413550
003F0258  ^ E9 D5FDFFFF jmp     003F0032
003F025D  ^ E9 5EFFFFFF jmp     003F01C0
003F0262  ^ E9 E0FFFFFF jmp     003F0247
003F0267  ^ E9 EAFEFFFF jmp     003F0156
003F026C    8D4C20 00  lea     ecx,[eax]
003F0270  ^ E9 34FEFFFF jmp     003F00A9
003F0275  ^ E9 48FEFFFF jmp     003F00C2
003F027A  ^ E9 6BFEFFFF jmp     003F00EA
003F027F  ^ E9 80FEFFFF jmp     003F0104
003F0284  ^ E9 CDFEFFFF jmp     003F0156
003F0289    6A 01      push    1
003F028B    C70424 EC1D400 mov  dword ptr [esp],401DEC
003F0292    FF2424     jmp     [esp]                Y esto ¿?
003F0295    0000       add     [eax],al
003F0297    0000       add     [eax],al
003F0299    0000       add     [eax],al
003F029B    0000       add     [eax],al
```

Pero qué es esto ¿? Qué sección más rara, si seguimos traceando llegamos a un salto que toma valor de OEP ;)

```
Address   Hex dump     Disassembly                  Comment
003F0289    6A 01      push    1
003F028B    C70424 EC1D400 mov  dword ptr [esp],401DEC
003F0292  - FF2424     jmp     [esp]                UnpackMe.00401DEC
003F0295    0000       add     [eax],al
003F0297    0000       add     [eax],al
003F0299    0000       add     [eax],al
003F029B    0000       add     [eax],al
003F029D    0000       add     [eax],al
003F029F    0000       add     [eax],al
003F02A1    0000       add     [eax],al
003F02A3    0000       add     [eax],al
003F02A5    0000       add     [eax],al
```

OEP==401DEC!!

Pues sí señores parece que al fin hemos llegado al OEP, jeje..

```
Address   Hex dump     Disassembly                  Comment
00401DEC    E8 0FE2FCFF call    003D0000
00401DF1    59         pop     ecx
00401DF2    85C0       test    eax,eax
00401DF4  v 75 08      jnz     short 00401DFE
00401DF6    6A 1C      push    1C
00401DF8    E8 C3000000 call    00401EC0
```

Este CALL está ROTO y hay Stolen Bytes!!

Bouuuuuuff!! Mereció la pena el esfuerzo, intenté sacar el OEP con trazas pero me resultó imposible, menos mal que esta API nos ayudó, de no ser así ya no sé qué hubiera hecho :D

Pero bien, lo importante es lo que tenemos, vamos a dumpear, pero antes veamos qué pasa con ese CALL 003D0000 que no nos gusta NADA, entremos (Enter):

Address	Hex dump	Disassembly	Comment
003D0000	33C0	xor eax,eax	
003D0002	6A 00	push 0	
003D0004	- E9 D0420300	jmp UnpackMe.004042D9	
003D0009	0000	add [eax],al	
003D000B	0000	add [eax],al	
003D000D	0000	add [eax],al	

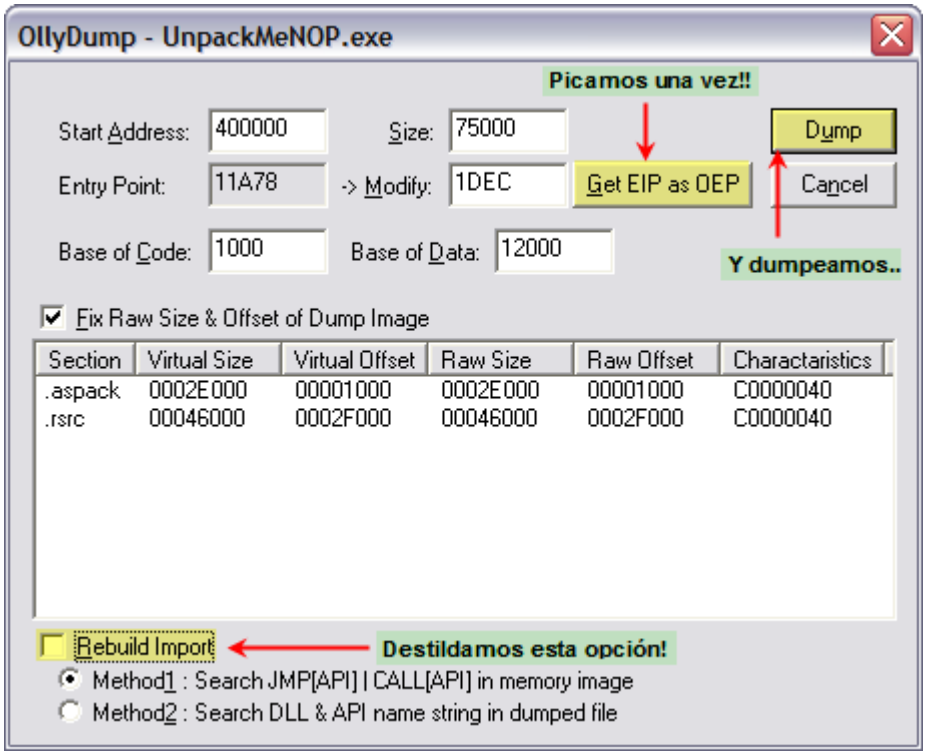
Ahí vemos que por suerte no hace gran cosa, si seguimos el JMP caemos aquí:

Address	Hex dump	Disassembly	Comment
004042D5	33C0	xor eax,eax	Líneas Robadas!
004042D7	6A 00	push 0	
004042D9	394424 08	cmp [esp+8],eax	Aqui el JMP!
004042DD	68 00100000	push 1000	
004042E2	0F94C0	sete al	
004042E5	50	push eax	
004042E6	FF15 EC204100	call [4120EC]	kernel32.HeapCreate
004042EC	85C0	test eax,eax	

Allí nos lleva (0x4042D9h) y por encima vemos las instrucciones que nos robó dicho CALL, por suerte fácil de reparar, simplemente hacemos esto:

Address	Hex dump	Disassembly	Comment
00401DEC	E8 E4240000	call 004042D5	Call reparado ;)
00401DF1	59	pop ecx	
00401DF2	85C0	test eax,eax	
00401DF4	v 75 08	jnz short 00401DFE	
00401DF6	6A 1C	push 1C	
00401DF8	E8 C3000000	call 00401EC0	
00401DFD	59	pop ecx	

Y ahora sí dumpeamos con OllyDump:



Las opciones son las que siempre solemos poner, destildar el REBUILD IMPORT ya que luego utilizaremos el ImpREC para reparar la IT+IAT.

NOTA: La IT es una tabla que guarda todo ejecutable donde están los nombres de las librerías y funciones que importa el programa, a partir de esta se forma

la IAT (en tiempo de carga) se hace por compatibilidad. (Para información más detallada ver los 4 tomos IMPORT TABLES A MANO del Maestro Ricardo)

Y nosotros dumpeamos, yo lo nombré tute.exe, pero si lo ejecutamos vemos que NO funciona, esto es lógico, primero que no tiene una IT (Import Table) ya que el packer se encargó de eliminarla. Pero ahora no podemos usar el Import Reconstructor (ImpREC) porque la IAT está dañada.

Afortunadamente hay métodos para recuperarla, vamos a ello..

-** Recuperando la IAT **-

Bueno esto espero sea más sencillo que todo lo anterior, de momento aprovechando que estamos en el OEP busquemos un salto a una función, este mismo por ejemplo (y nos situamos sobre él):

Address	Hex dump	Disassembly		Comment
00401E11	8975 FC	mov	[ebp-4],esi	
00401E14	E8 00230000	call	00404119	
00401E19	FF15 A8204100	call	[4120A8]	kernel32.GetCommandLineA
00401E1F	A3 FCA74100	mov	[41A7FC],eax	

Hacemos un FOLLOW IN DUMP-MEMORY ADDRESS y estamos en plena IAT, desde aquí subimos hacia arriba con la rueda del ratón y encontramos su inicio:

Address	Hex dump	ASCII
00412000	C0 32 3E 00 F8 32 3E 00 30 33 3E 00 68 33 3E 00	└2>.°2>.03>.h3>.
00412010	00 00 00 00 A0 33 3E 00 00 00 00 00 18 2B 3E 00	....á3>.....↑+>.
00412020	50 APIs MALAS ;(3 3E 00 C0 2B 3E 00 F8 2B 3E 00	P+>.ê+>.└+>.°+>.
00412030	30 2C 3E 00 68 2C 3E 00 A0 2C 3E 00 D8 2C 3E 00	0,>.h,>.á,>.Ï,>.
00412040	10 2D 3E 00 48 2D 3E 00 80 2D 3E 00 B8 2D 3E 00	►->.H->.Ç->.@->.
00412050	F0 2D 3E 00 28 2E 3E 00 60 2E 3E 00 98 2E 3E 00	-->.(.>.'.>.ÿ.>.
00412060	D0 2E 3E 00 08 2F 3E 00 40 2F 3E 00 78 2F 3E 00	§.>.◘/>.@/>.x/>.
00412070	B0 2F 3E 00 E8 2F 3E 00 20 30 3E 00 58 30 3E 00	▒/>.Φ/>. 0>.X0>.
00412080	90 30 3E 00 C8 30 3E 00 00 31 3E 00 38 31 3E 00	É0>.╚0>..1>.81>.
00412090	70 31 3E 00 A8 31 3E 00 E0 31 3E 00 00 00 00 00	p1>.¿1>.Ó1>.....
004120A0	44 0C F5 77 82 17 E4 77 88 D3 E4 77 5D 29 E5 77	D.§wé‡ßwêËßw]]ßw
004120B0	BC 16 E4 77 6B 1[illegible] F4 77 A1 16 F4 77 B0 AA E5 77	╝_ßwk§¶wí_¶w▒¬ßw
004120C0	5F 8C F4 77 40 1A F4 77 F7 8B E4 77 E7 55 E7 77	_î¶w@→¶w≈ïßwÞUÞw
004120D0	EC 87 E7 77 46 05 E5 77 97 2B E6 77 78 D3 E4 77	ÿçÞwF♣ßwù+µwxÊßw
004120E0	46 F8 E4 77 E1 E6 E4 77 8D C2 E4 77 2B C2 E4 77	F°ßwÞµßwíτßw+τßw
004120F0	5D A2 E4 77 04 8F E4 77 APIs BUENAS ;)	24 E8 77	]ßßw♦Aßwßßßw=§Þw
00412100	A3 76 E5 77 12 A3 E4 77 96 1C E6 77 7C BF E4 77	úvßw‡úßwûLµw¡τßw
00412110	14 9C E4 77 F7 9D E5 77 16 C9 E5 77 B7 BE E4 77	¶£ßw≈Øßw▬ßßwÀ¥ßw
00412120	58 EF E4 77 6F C9 E4 77 14 9F E5 77 44 E6 E4 77	X'ßwo╔ßw¶ƒßwDµßw
00412130	2C A2 E5 77 4B C0 E4 77 02 15 F4 77 B1 98 E4 77	,ßßwK└ßw☻§¶w▒ÿßw

Bueno ahí tenemos el primer dato de tabla:

Inicio: 412000

Ya que estamos busquemos el final:

Address	Hex dump	ASCII
00412330	A8 23 3E 00 E0 23 3E 00 18 24 3E 00 50 24 3E 00	¿#>.Ó#>.↑$>.P$>.
00412340	88 24 3E 00 C0 24 3E 00 F8 24 3E 00 30 25 3E 00	ê$>.└$>.°$>.0%>.
00412350	68 25 3E 00 A0 25 3E 00 D8 25 3E 00 10 26 3E 00	h%>.á%>.Ï%>.►&>.
00412360	48 26 3E 00 80 26 3E 00 B8 26 3E 00 F0 26 3E 00	H&>.Ç&>.@&>.-&>.
00412370	28 27 3E 00 60 27 3E 00 98 27 3E 00 D0 27 3E 00	('>.''>.ÿ'>.§'>.
00412380	08 28 3E 00 40 28 3E 00 78 28 3E 00 B0 28 3E 00	◘(>.@(>.x(>.▒(>.
00412390	E8 28 3E 00 20 29 3E 00 58 29 3E 00 90 29 3E 00	Φ(>.)>.X)>.É)>.
004123A0	C8 29 3E 00 00 2A 3E 00 38 2A 3E 00 70 2A 3E 00	╚)>..*>.8*>.p*>.
004123B0	A8 2A 3E 00 E0 2A 3E 00 00 00 00 00 AD 5F F8 72	¿*>.Ó*>.....¡_°r
004123C0	63 16 F8 72 69 5E F8 72 00 00 00 00 00 00 00 00	c_°ri^°r.....
004123D0	E8 25 41 00 D8 23 41 00 11 01 00 00 00 00 00 00	Φ%A.Ï#A.◄Ø.....
004123E0	46 E1 00 00 46 E1 00 00 0C 00 00 00 54 F4 40 00	FÞ..FÞ.....T¶@.
004123F0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....

Final: 4123C8

Y rápido sacamos su largo restando ambas:

Largo: 3C8

Ahora que tenemos la IAT delimitada vamos a tratar de evitar que redireccione los punteros, elijamos uno MALO como [4121FC]==003E12D0h que haciendo pruebas descubrí que es el primero en redireccionarse.

Ahora reiniciamos Olly pues el dumpeado ya lo hicimos, coloquemos un BPM ON MEMORY WRITE en la entrada [4121FC] y damos RUN (quitar todos los BP y HBP puestos).Si trabajamos con el crackme modificado no olvidar forzar el salto de 0x3B314Dh además de NOPear el DIV EAX de 0x3B03FBh, saldrá la nag que aceptamos y Olly nos para en un MOVS pero no guarda la API, vuelvo a darle RUN para llegar a la zona buscada, y es a la segunda donde la guarda, aquí:

```
Address   Hex dump     Disassembly                          Comment
003B6AD3  891F         mov     [edi],ebx                    Aqui!!!!
003B6AD5  EB 02        jmp     short 003B6AD9
003B6AD7  8907         mov     [edi],eax
003B6AD9  8185 D9274100 add    dword ptr [ebp+4127D9],38
003B6AE3  8385 40944100 add    dword ptr [ebp+419440],4
003B6AEA  61           popad
003B6AEB  80BD 44944100 cmp    byte ptr [ebp+419444],0
003B6AF2  75 0C        jnz     short 003B6B00
```

En EDI está un puntero a laIAT y en EBX la API encriptada, lo mejor de todo es ese POPAD que asoma por abajo pues al superarlo nos deja ver la API, jeje..

```
Registers (3DNow!)
EAX 77D19265 User32.ScreenToClient
ECX 77F432FA ntdll.77F432FA
EDX 77FB306C ntdll.77FB306C
EBX 00415F2A ASCII "ScreenToClient"
ESP 0012F218
EBP FFF9D827
ESI 00415034 UnpackMe.00415034
EDI 004121FC UnpackMe.004121FC
```

La tenemos en EAX, podemos hacer un script o bien injertar, tiramos por esta segunda opción (injertar) porque la veo más rápida, simplemente cambiamos la comparación donde estamos por un JMP a una zona libre:

```
Address   Hex dump     Disassembly                          Comment
003B6AD3  891F         mov     [edi],ebx                    Aqui!!!!
003B6AD5  EB 02        jmp     short 003B6AD9
003B6AD7  8907         mov     [edi],eax
003B6AD9  8185 D9274100 add    dword ptr [ebp+4127D9],
003B6AE3  8385 40944100 add    dword ptr [ebp+419440],
003B6AEA  61           popad
003B6AEB  E9 701F0000  jmp     003B8A60                     JMP ensamblado!
003B6AF0  90           nop
003B6AF1  90           nop
003B6AF2  75 0C        jnz     short 003B6B00
```

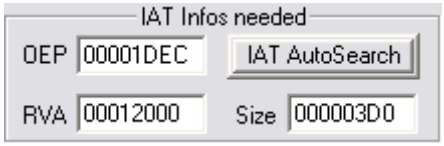
Ahora vamos a esa zona y hacemos simplemente esto:

```
Address   Hex dump     Disassembly                          Comment
003B8A60  8907         mov     [edi],eax
003B8A62  80BD 44944100 cmp    byte ptr [ebp+419444],0
003B8A69  68 F26A3B00  push    3B6AF2
003B8A6E  C3           ret
003B8A6F  90           nop
003B8A70  0000         add     [eax],al
003B8A72  0000         add     [eax],al
003B8A74  0000         add     [eax],al
```

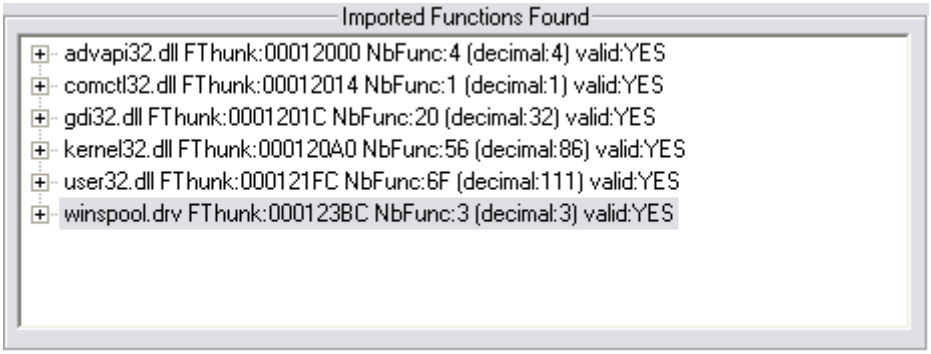
Unicamente salvamos la API correcta en la IAT, reemplazamos la instrucción sobrescrita y volvemos, así de fácil (sin complicarnos la vida =) y como ya tenemos el dumpeado nos da igual llegar al OEP o no, simplemente RUN..

```
Address   Hex dump                                         ASCII
00412000  B1 63 DB 77 D8 17 DA 77 9A 22 DA 77 D6 27 DA 77  ▒c■wï‡┌wÜ"┌wÏ'┌w
00412010  00 00 00 00 14 48 31 77 00 00 00 00 6F 59 00 7F  ....¶H1w....oY.⌂
00412020  B3 5E 00 7F BE 8F 00 7F EA 68 00 7F 51 B4 00 7F  |^.⌂¥A.⌂ûh.⌂Q┤.⌂
00412030  A3 E9 00 7F 8A 20 02 7F A2 E8 00 7F 85 DD 02 7F  úÚ.⌂è ☻⌂óþ.⌂à¦☻⌂
00412040  23 67 00 7F 60 5B 00 7F B3 75 00 7F 1D 8B 00 7F  #g.⌂'[.⌂|u.⌂#ï.⌂
00412050  44 61 00 7F B0 38 03 7F 95 6A 00 7F CB D1 00 7F  Da.⌂▒8♥⌂ò j.⌂╥Ð.⌂
00412060  2D BE 00 7F CB 7A 01 7F A2 94 00 7F 17 94 00 7F  -¥.⌂╥z☺⌂óö.⌂‡ö.⌂
00412070  21 7A 00 7F EA 90 00 7F E2 5D 00 7F 37 5D 00 7F  !z.⌂ÜÉ.⌂ô].⌂7].⌂
00412080  56 78 00 7F 06 98 00 7F CA A3 00 7F F3 C4 00 7F  Vx.⌂♠ÿ.⌂╧ú.⌂%─.⌂
00412090  C2 77 00 7F A8 60 00 7F 56 61 00 7F 00 00 00 00  ┬w.⌂¿'.⌂Va.⌂....
004120A0  44 0C F5 77 82 17 E4 77 88 D3 E4 77 5D 29 E5 77  D.§wé‡õwëÉõw])õw
004120B0  BC 16 E4 77 6B 15 F4 77 A1 16 F4 77 B0 AA E5 77  ╝_õwk§¶wí_¶w▒¬õw
004120C0  5F 8C F4 77 40 1A F4 77 F7 8B E4 77 E7 55 E7 77  _î¶w@→¶w-ïõwÞÏÞw
004120D0  EC 87 E7 77 46 05 E5 77 97 2B E6 77 78 D3 E4 77  ÿÇÞwF♣õwù+µwxËõw
004120E0  46 F8 E4 77 E1 E6 E4 77 8D C2 E4 77 2B C2 E4 77  F°õwßµõwi┬õw+┬õw
```

Vemos que la IAT quedó perfecta, abrimos ImpREC sin cerrar este Olly y attcheamos el proceso de la lista ATTACH TO AN ACTIVE PROCESS, le damos el falso OEP de momento (luego lo cambiaremos), son estos datos a rellenar:



Luego a GET IMPORTS y allí las vemos TODAS ;)

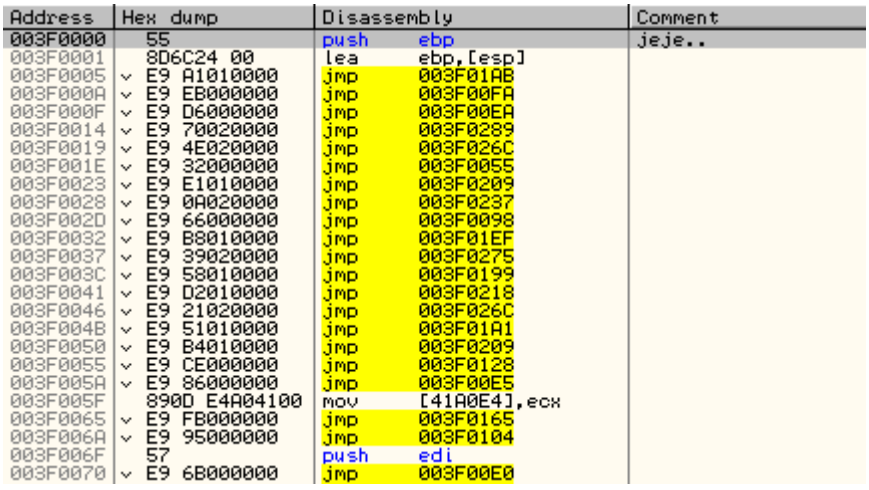


Ahora FIX DUMP y elegimos tute.exe, nos dejará tute_.exe que ahora sí tiene la IT reparada, jeje..

Y bien lo único que nos queda son los Stolen Bytes, vamos por ellos..

-** Buscando los Stolen Bytes **-

Pues si todo va bien este será el último paso del desempakado y uno de los más sencillos, recordemos que entes GetVersion nos dejó en una sección "rara" pues bien si miramos al inicio de esta vemos:



Address	Hex dump	Disassembly		Comment
003F0000	55	push	ebp	jeje..
003F0001	8D6C24 00	lea	ebp,[esp]	
003F0005	E9 A1010000	jmp	003F01AB	
003F000A	E9 EB000000	jmp	003F00FA	
003F000F	E9 D6000000	jmp	003F00EA	
003F0014	E9 70020000	jmp	003F0289	
003F0019	E9 4E020000	jmp	003F026C	
003F001E	E9 32000000	jmp	003F0055	
003F0023	E9 E1010000	jmp	003F0209	
003F0028	E9 0A020000	jmp	003F0237	
003F002D	E9 66000000	jmp	003F0098	
003F0032	E9 B8010000	jmp	003F01EF	
003F0037	E9 39020000	jmp	003F0275	
003F003C	E9 58010000	jmp	003F0199	
003F0041	E9 D2010000	jmp	003F0218	
003F0046	E9 21020000	jmp	003F026C	
003F004B	E9 51010000	jmp	003F01A1	
003F0050	E9 B4010000	jmp	003F0209	
003F0055	E9 CE000000	jmp	003F0128	
003F005A	E9 86000000	jmp	003F00E5	
003F005F	890D E4A04100	mov	[41A0E4],ecx	
003F0065	E9 FB000000	jmp	003F0165	
003F006A	E9 95000000	jmp	003F0104	
003F006F	57	push	edi	
003F0070	E9 6B000000	jmp	003F00E0	

Pues aquí comienzan los Stolen Bytes y recuperarlos es bien sencillo, sólo hay que poner un BP al inicio de la zona cuando se crea, los pasos a seguir están bien detallados (antes los vimos). Una vez aquí sólo hay que ejecutar anotando TODAS las instrucciones que ejecute, de esta manera podremos reconstruir después el dumpeado pues no tiene estos bytes.

Aquí hago una correlación de lo que ejecutó, los JMPs no son tomados como stolen bytes ya que no afectan a la pila y/o registros ok? Estas son..

```
003F0000     55               push    ebp
003F0001     8D6C24 00        lea     ebp,[esp]              ; PUSH EBP!!
003F0251     6A FF            push    -1
003F0253     68 50354100      push    413550
003F01EF     68 1C434000      push    40431C
003F01F4     64:A1 00000000   mov     eax,fs:[0]
003F0241     50               push    eax
003F009D     64:8925 0000000>mov      fs:[0],esp
003F0199     83C4 A8          add     esp,-58
003F00F4     53               push    ebx
003F01B0     56               push    esi
003F006F     57               push    edi
003F0218     8965 E8          mov     [ebp-18],esp
003F021B     FF15 A0214100    call    [4121A0]             ; kernel32.GetVersion
003F0221     33D2             xor     edx,edx
003F00C2     8AD4             mov     dl,ah
003F016A     8915 ECA04100    mov     [41A0EC],edx
003F026C     8D4C20 00        lea     ecx,[eax]
003F0175     81E1 FF000000    and     ecx,0FF
003F013C     890D E8A04100    mov     [41A0E8],ecx
003F0113     69C9 00010000    imul    ecx,ecx,100
003F01C0     03CA             add     ecx,edx
003F005F     890D E4A04100    mov     [41A0E4],ecx
003F00C9     C1E8 10          shr     eax,10
003F0209     A3 E0A04100      mov     [41A0E0],eax
003F0289     6A 01            push    1
003F028B     C70424 EC1D4000 mov      dword ptr [esp],401>
003F0292   - FF2424           jmp     [esp]                ; UnpackMe.00401DEC
```

Las últimas líneas no las contamos pues se está preparando para saltar al OEP, seleccionamos las demás y las pegamos sobre el falso OEP:

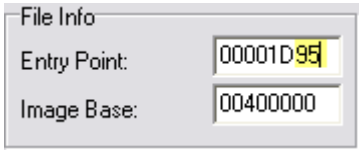
```
Address   Hex dump        Disassembly                          Comment
00401D95  55              push    ebp
00401D96  8BEC            mov     ebp,esp
00401D98  6A FF           push    -1
00401D9A  68 50354100     push    00413550
00401D9F  68 1C434000     push    0040431C
00401DA4  64:A1 00000000  mov     eax,fs:[0]
00401DAA  50              push    eax
00401DAB  64:8925 000000( mov     fs:[0],esp
00401DB2  83C4 A8         add     esp,-58
00401DB5  53              push    ebx
00401DB6  56              push    esi
00401DB7  57              push    edi
00401DB8  8965 E8         mov     [ebp-18],esp
00401DBB  FF15 A0214100   call    [<&kernel32.GetVersion>]     kernel32.GetVersion
00401DC1  33D2            xor     edx,edx
00401DC3  8AD4            mov     dl,ah
00401DC5  8915 ECA04100   mov     [41A0EC],edx
00401DCB  8D08            lea     ecx,[eax]
00401DCD  81E1 FF000000   and     ecx,0FF
00401DD3  890D E8A04100   mov     [41A0E8],ecx
00401DD9  69C9 00010000   imul    ecx,ecx,100
00401DDF  03CA            add     ecx,edx
00401DE1  890D E4A04100   mov     [41A0E4],ecx
00401DE7  C1E8 10         shr     eax,10
00401DEA  6A 01           push    1
00401DEC  E8 E4240000     call    004042D5
00401DF1  59              pop     ecx
```

No sé porqué pero tuve que dejar una instrucción fuera pues no cabía:

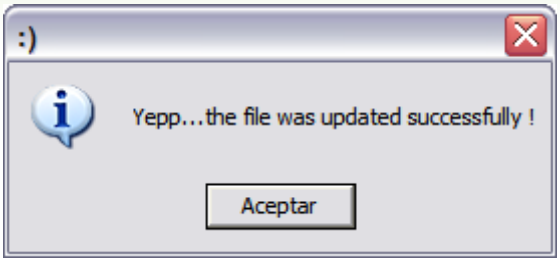
```
003F0209     A3 E0A04100      mov     [41A0E0],eax
```

Seleccionamos desde el primero al último y con el botón derecho picamos sobre esta zona, elegimos COPY TO EXECUTABLE-SELECTION, se abrirá otra pantallita donde elegimos SAVE FILE.

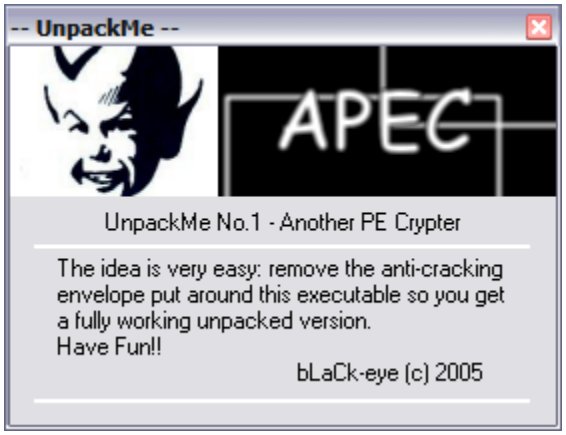
Si lo ejecutamos NO funciona, claro el OEP! Le hemos colocado los Stolen Bytes pero aún arranca desde el falso OEP, esto lo arreglamos con PEditor 1.7 así:



Damos a APPPLY CHANGES y sale este cartel:



Bueno cerremos PEditor y ejecutemos tute_.exe..



Jejejeje.. allí está corriendo desempakado :D

Y bueno hasta aquí llegó este UnPackME de bLaCk-eye (muy entretenido =). Sólo Espero que os haya gustado el tute, espero poder escribir muchísimos más :D:D

-** Saludos y/o Agradecimientos **-

Y bueno, a TODA la lista CracksLatinoS y la banda el yahoo en especial, como solid, Ricardo, Arapumk, RedH@wk, Absolom, Suei, Dapaf, Morales, Mintaka, Emadicius, Juan Jose, Mechnos, Lionel, Rdgmax, _primax, Silver, Agujeta, Cristian Padyck, k0rt, Alfonso_iat, Carlos, txino_crk, DeAth, , marciano, Ulaterck, MairJr Sofá, c40s, Yllera, +NCR, Hiei y un largo etcétera..

A los autores de todas las tools aquí utilizadas:

- Oleh Yuschuk por OllyDbg 1.10
- MackT por Import Reconstructor v1.6f
- rdgmax por RDG Packer Detector v0.6.5 Beta
- Yoda por PEditor 1.7
- etc…

Y bueno, poco más que decir de un UnPackMe muy entretenido a mi parecer, y que me gustó mucho desempakar, espero que no sea el último ;)

-** Contacto **-

El autor puede ser contactado mediante esta dirección de correo electrónico:

syxe05@gmail.com